CS T42 MICROPROCESSOR AND MICROCONTROLLER 2 MARKS Q&A

Y2/S4

UNIT I

Intel 8085 Microprocessor: Introduction - Need for Microprocessors – Evolution – Intel 8085 Hardware - Architecture – Pin description - Internal Registers – Arithmetic and Logic Unit – Control Unit – Instruction word size - Addressing modes – Instruction Set – Assembly Language Programming - Stacks and Subroutines - Timing Diagrams. Evolution of Microprocessors – 16-bit and 32-bit microprocessors.

1. What is microprocessor? ****

A microprocessor is a multipurpose, programmable, register based; clock-driven, electronic device that reads binary instructions from a storage device called memory accepts binary data as input and processes data according to those instructions and provides result as output. The power supply of 8085 is +5V and clock frequency in 3MHz.

2. Define ROM?

A memory that stores binary information permanently. The information can be read from this memory but cannot be altered.

3. **List few applications of microprocessor-based system. ******

It is used:

- For measurements, display and control of current, voltage, temperature, pressure, etc.
- For traffic control and industrial tool control.
- For motor speed control of machines.
- For data acquisition system etc.

4. What is an Assembler?

A computer program that translate an assembly language program from mnemonics to the binary machine code of a computer.

5. What are the four primary operations of a MPU? **

1. Memory read
2. Memory write
3. I/O read
4. I/O write

6. What do you mean by address bus? ***

A group of lines that are used to send a memory address or a device address from the MPU to the memory location or a peripheral. The 8085 microprocessor has 16 address lines.

7. How many memory locations can be addressed by a microprocessor with 14 address lines?

The 8085 MPU with its 14-bit address is capable of addressing 214=16,384 (ie) 16K memory locations.

8. Why is the data bus bi-directional?

The data bus is bi-directional because the data flow in both directions between the MPU and memory and peripheral devices.

9. What is the function of the accumulator? ***

The accumulator is the register associated with the ALU operations and sometimes I/O operation s. It is an integral part of ALU. It holds one of data to be processed b y ALU. It also temporarily stores the result of the operation performed by the ALU.

10. Define control bus? *

This is single line that is generated by the MPU to provide timing of various operations.

11. What is a flag? Write the flags of 8085. ***

The data conditions, after arithmetic or logical operations, are indicated by setting or resetting the flip-flops called flags.

The 8085 has nine flags and they are

1. Carry Flag (CF)
2. Parity Flag (PF)
3. Auxiliary carry Flag (AF)
4. Zero Flag (ZF)
5. Sign Flag (SF)

12. Why are the program counter and the stack pointer 16-bit registers? **

Memory locations for the program counter and stack pointer have 16-dit address. So the PC and SP have 16-bit registers.

13. Define memory word?

The number of bits stored in a register is called a memory word.

14. Explain the function of ALU and IO/M signals in the 8085 architecture? **

The ALU signal goes high at the beginning of each machine cycle indicating the availability of the address on the address bus, and the signal is used to latch the low-order address bus. The IO/M signal is a status signal indicating whether the machine cycle is I/O or memory operation. The IO/M signal is combined with the RD and WR control signals to generate IOR, IOW, MEMW, MEMR.

15. Write down the control and status signals? **

- Two control signals and three status signals
- Control signals: RD and WR
- Status signals: IO/M, S1, and S2

16. Define machine cycle? **

Machine cycle is defined, as the time required completing one operation of accessing memory, I/O, or acknowledging an external request.

17. Define T-state? *

T-state is defined as one subdivision of the operation of performed in one clock period.

18. Give the bit positions reserved for the flags? *

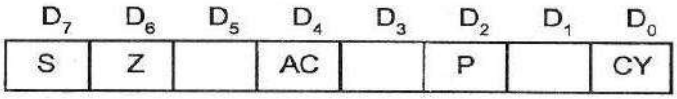


Fig 1.7 : Bit positions of various flags in the flag register of 8085

19. Define instruction cycle? *

Instruction cycle is defined, as the time required completing the execution of the instruction.

20. List the allowed register pairs of 8085.

- B-C register pair
- D-E register pair
- H-L register pair

21. Mention the purpose of SID and SOD lines *

- **SID (Serial input data line):**
- It is an input line through which the microprocessor accepts serial data.
- **SOD (Serial output data line):**
- It is an output line through which the microprocessor sends output serial data.

22. What is an Opcode?

The part of the instruction that specifies the operation to be performed is called the operation code or opcode.

23. What is the function of IO/M signal in the 8085?

It is a status signal. It is used to differentiate between memory locations and I/O operations. When this signal is low (IO/M = 0) it denotes the memory related operations. When this signal is high (IO/M = 1) it denotes an I/O operation.

24. List out the five categories of the 8085 instructions. Give examples of the instructions for each group. *

- Data transfer group – MOV, MVI, LXI.
- Arithmetic group – ADD, SUB, INR.
- Logical group –ANA, XRA, CMP.

- Branch group – JMP, JNZ, CALL
- Stack I/O and Machine control group – PUSH, POP, IN, HLT.

25. Explain the difference between a JMP instruction and CALL instruction. **

A JMP instruction permanently changes the program counter. A CALL instruction leaves information on the stack so that the original program execution sequence can be resumed.

26. Explain the purpose of the I/O instructions IN and OUT.

The IN instruction is used to move data from an I/O port into the accumulator. The OUT instruction is used to move data from the accumulator to an I/O port. The IN & OUT instructions are used only on microprocessor, which use a separate address space for interfacing.

27. What is the difference between the shifts and rotate instructions?

A rotate instruction is a closed loop instruction. That is, the data moved out at one end is put back in at the other end. The shift instruction loses the data that is moved out of the last bit locations.

28. What is meant by Wait State? *

This state is used by slow peripheral devices. The peripheral devices can transfer the data to or from the microprocessor by using READY input line. The microprocessor remains in wait state as long as READY line is low. During the wait state, the contents of the address, address/data and control buses are held constant.

29. Define instruction cycle, machine cycle and T-state **

Instruction cycle is defined, as the time required completing the execution of an instruction. Machine cycle is defined as the time required completing one operation of accessing memory, I/O or acknowledging an external request. T-cycle is defined as one subdivision of the operation performed in one clock period.

30. What is the use of ALE **

The ALE is used to latch the lower order address so that it can be available in T2 and T3 and used for identifying the memory address. During T1 the ALE goes high, the latch transparent ie, the output changes according to the input data, so the output of the latch is the lower order address. When ALE goes low the lower order address is latched until the next ALE.

31. How many machine cycles does 8085 have, mention them ***

The 8085 have seven machine cycles. They are

- Opcode fetch
- Memory read
- Memory write
- I/O read
- I/O write
- Interrupt acknowledge
- Bus idle

32. Explain the signals HOLD, READY and SID. *

HOLD indicates that a peripheral such as DMA controller is requesting the use of address bus, data bus and control bus. READY is used to delay the microprocessor read or write cycles until a slow responding peripheral is ready to send or accept data. SID is used to accept serial data bit by bit.

33. Explain LDA, STA and DAA instructions

LDA copies the data byte into accumulator from the memory location specified by the 16-bit address. STA copies the data byte from the accumulator in the memory location specified by 16-bit address. DAA changes the contents of the accumulator from binary to 4-bit BCD digits.

**34. Explain the different instruction formats with examples. **

The instruction set is grouped into the following formats

- One byte instruction MOV C,A
- Two byte instruction MVI A,39H
- Three byte instruction JMP 2345H

35. What is the use of addressing modes, mention the different types

The various formats of specifying the operands are called addressing modes, it is used to access the operands or data. The different types are as follows

- Immediate addressing
- Register addressing
- Direct addressing
- Indirect addressing
- Implicit addressing

36. What is the use of bi-directional buffers?

It is used to increase the driving capacity of the data bus. The data bus of a microcomputer system is bi-directional, so it requires a buffer that allows the data to flow in both directions.

37. Define stack and explain stack related instructions **

The stack is a group of memory locations in the R/W memory that is used for the temporary storage of binary information during the execution of the program. The stack related instructions are PUSH & POP.

38. Compare CALL and PUSH instructions *****

CALL	PUSH
When CALL is executed the microprocessor automatically stores the 16-bit address of the instruction next to CALL on the stack	The programmer uses the instruction PUSH to save the contents of the register pair on the stack
When CALL is executed the stack pointer is decremented by two	When PUSH is executed the stack pointer register is decremented by two.

39. What is Microcontroller and Microcomputer. **

Microcontroller is a device that includes microprocessor; memory and I/O signal lines on a single chip, fabricated using VLSI technology. Microcomputer is a computer that is designed using microprocessor as its CPU. It includes microprocessor, memory and I/O.

40. Compare RET and POP. ****

RET	POP
RET transfers the contents of the top two locations of the stack to the PC	POP transfers the contents of the top two locations of the stack to the specified register pair
When RET is executed the SP is incremented by two	When POP is executed the SP is incremented by two
Has 8 conditional RETURN instructions	No conditional POP instructions

41. What is assembly language programming (ALP)? List its field.

Assembly language programming is a program written in a mnemonics or set of instruction.

ALP contain 4 fields,

- Label
- Opcode
- Operand
- Comments

Unit: I

Intel 8085 Microprocessor: Introduction - Need for Microprocessors – Evolution – Intel 8085 Hardware - Architecture – Pin description - Internal Registers – Arithmetic and Logic Unit – Control Unit – Instruction word size - Addressing modes – Instruction Set – Assembly Language Programming - Stacks and Subroutines - Timing Diagrams - Evolution of Microprocessors – 16-bit and 32 - bit microprocessors.

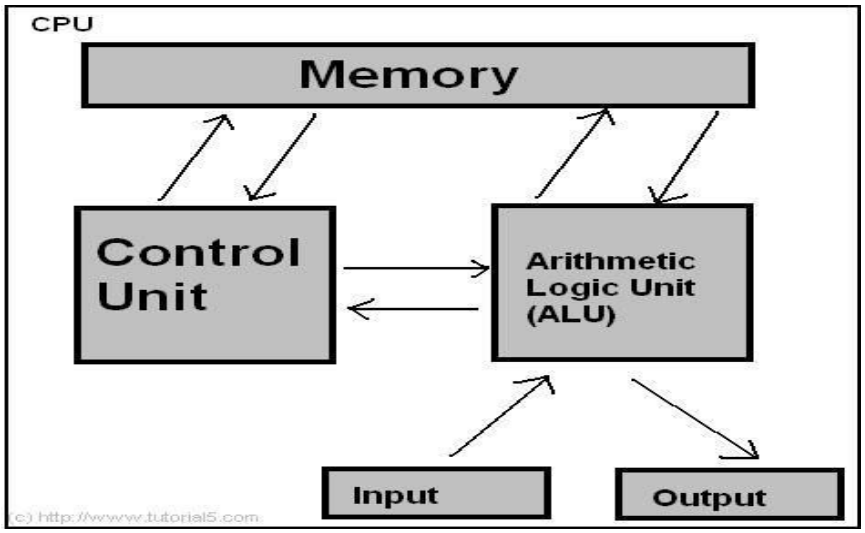
---

1. Introduction

Definition of the Microprocessor:

The microprocessor is a programmable device that takes in numbers, performs on them arithmetic or logical operations according to the program stored in memory and then produces other numbers as a result.

"CPU is in the form of chip". The major component of microprocessor is CPU, memory, input and output devices.



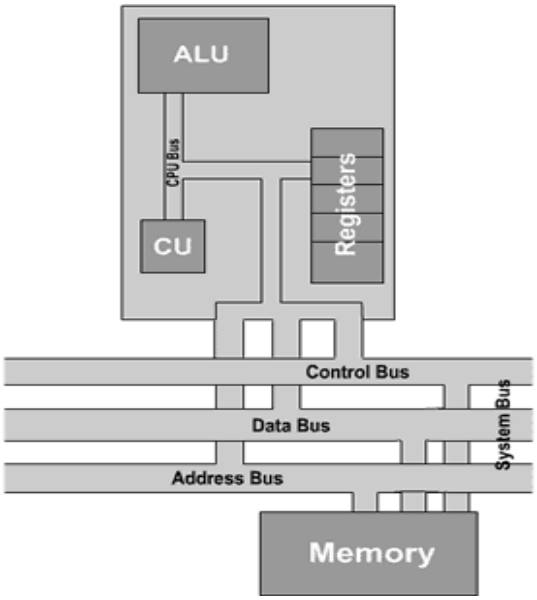
Basic Concepts of Microprocessors & Differences between:

Microcomputer –a computer with a microprocessor as its CPU. Includes memory, I/O etc.,

Microprocessor –silicon chip which includes ALU, register circuits & control circuits.

Microcontroller –silicon chip which includes microprocessor, memory & I/O in a single package.

1.1 GENERAL ARCHITECTURE OF MICROPROCESSOR



Organization of Microprocessor

The microprocessor is sometimes referred to as the 'brain' of the personal computer, and is responsible for the processing of the instructions which make up computer software. It houses the central processing unit, commonly referred to as the CPU.

CPU Structure This section, using a simplified model of a central processing unit as an example, takes you through the role of each of the major constituent parts of the CPU.

The simplified model consists of five parts, which are:

Arithmetic & Logic Unit (ALU) The part of the central processing unit that deals with operations such as addition, subtraction, and multiplication of integers and Boolean operations. It receives control signals from the control unit telling it to carry out these operations. For more, click the title above.

Control Unit (CU) This controls the movement of instructions in and out of the processor, and also controls the operation of the ALU. It consists of a decoder, control logic circuits, and a clock to ensure everything happens at the correct time. It is also responsible for performing the instruction execution cycle.

Register Array This is a small amount of internal memory that is used for the quick storage and retrieval of data and instructions. All processors include some common registers used for specific functions, namely the program counter, instruction register, accumulator, memory address register and stack pointer.

System Bus This is comprised of the control bus, data bus and address bus. It is used for connections between the processor, memory and peripherals, and transfers of data between the various parts.

Memory The memory is not an actual part of the CPU itself, and is instead housed elsewhere on the motherboard. However, it is here that the program being executed is stored, and as such is a crucial part of the overall structure involved in program execution.

2. EVOLUTION OF MICROPROCESSORS

- **4-bit Microprocessors**

The first microprocessor was introduced in 1971 by Intel Corp. It was named Intel 4004 as it was a 4 bit processor. It was a processor on a single chip. It could perform simple arithmetic and logic operations such as addition, subtraction, boolean AND and boolean OR. It had a control unit capable of performing control functions like fetching an instruction from memory, decoding it, and generating control pulses to execute it. It was able to operate on 4 bits of data at a time. This first microprocessor was quite a success in industry. Soon other microprocessors were also introduced. Intel introduced the enhanced version of 4004, the 4040. Some other 4 bit processors are International's PPS4 and Thoshiba's T3472.

- **8-bit Microprocessors**

The first 8 bit microprocessor which could perform arithmetic and logic operations on 8 bit words was introduced in 1973 again by Intel. This was Intel 8008 and was later followed by an improved version, Intel 8088. Some other 8 bit processors are Zilog-80 and Motorola M6800.

- **16-bit Microprocessors**

The 8-bit processors were followed by 16 bit processors. They are Intel 8086 and 80286.

- **32-bit Microprocessors**

The 32 bit microprocessors were introduced by several companies but the most popular one is Intel 80386.

- **Pentium Series**

Instead of 80586, Intel came out with a new processor namely Pentium processor. Its performance is closer to RISC performance. Pentium was followed by Pentium Pro CPU. Pentium Pro allows allow multiple CPUs in a single system in order to achive multiprocessing. The MMX extension was added to Pentium Pro and the result was Pentiuum II. The low cost version of Pentium II is celeron.

The Pentium III provided high performance floating point operations for certain types of computations by using the SIMD extensions to the instruction set. These new instructions make the Pentium III faster than high-end RISC CPUs.

We divide the years of development of microprocessors as 5 generations.

- **First generation (1971 – 73)**

Intel Corporation introduced 4004, the first microprocessor in 1971. It is evolved from the development effort while designing a calculator chip.

There were three other microprocessors in the market during the same period:

- Rockwell International's PPS-4 (4 bits)
- Intel's 8008 (8 bits)
- National Semiconductor's IMP-16 (16 bits)

They were fabricated using PMOS technology which provided low cost, slow speed and low output currents. They were not compatible with TTL.

- **Second Generation (1974 – 1978)**

Some of the popular processors were:

- Motorola's 6800 and 6809
- Intel's 8085
- Zilog's Z80

They were manufactured using NMOS technology. This technology offered faster speed and higher density than PMOS It is TTL compatible

- **Third generation microprocessors (1979 – 80)**

This age is dominated by 16 – bits microprocessor some of them were:

- Intel's 8086/80186/80286
- Motorola's 68000/68010

They were designed using HMOS technology HMOS provides some advantages over NMOS as Speed-power-product of HMOS is four times better than that of NMOS HMOS can accommodate twice the circuit density compared to NMOS Intel used HMOS technology to recreate 8085A and named it as 8085AH with a higher price tag.

- **Fourth Generation (1981 – 1995)**

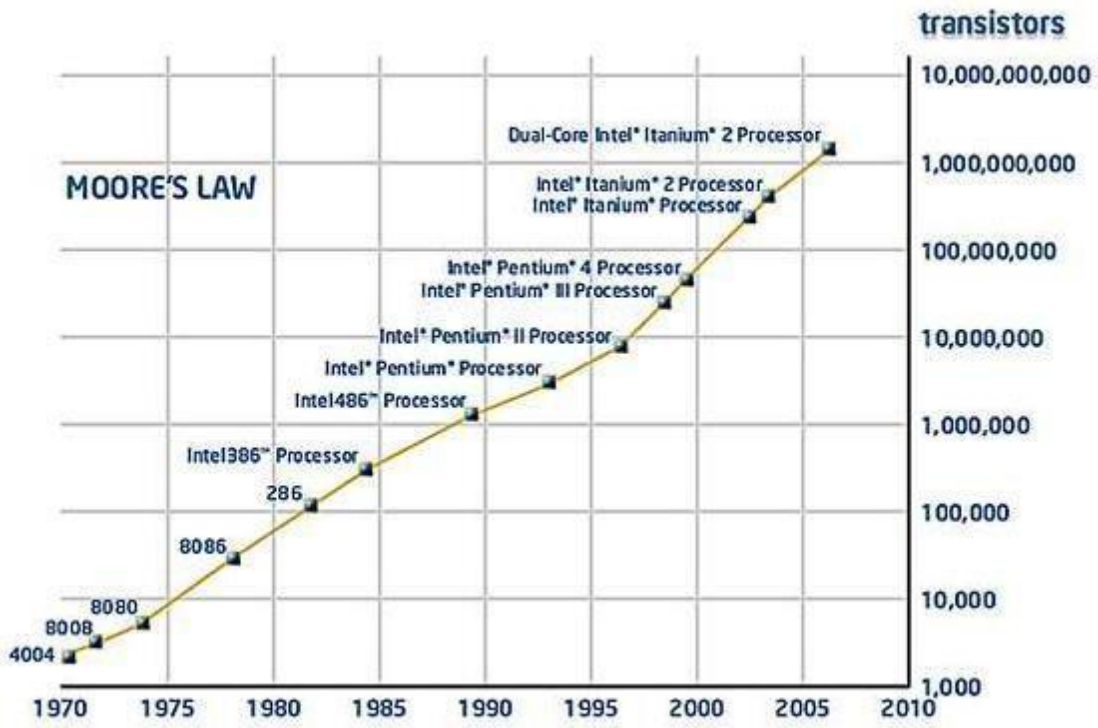
- This era marked the beginning of 32 bits microprocessors.
- Intel introduced 432, which was bit problematic
- Then a clean 80386 in launched.
- Motorola introduced 68020/68030.

They were fabricated using low-power version of the HMOS technology called HCMOS.

Motorola introduced 32-bit RISC processors called MC88100.

- **Fifth Generation (1995 – till date)**

This age the emphasis is on introducing chips that carry on-chip functionalities and improvements in the speed of memory and I/O devices along with introduction of 64-bit microprocessors. Intel leads the show here with Pentium, Celeron and very recently dual and quad core processors working with up to 3.5GHz speed.



3. HARDWARE ARCHITECTURE OF INTEL -8085

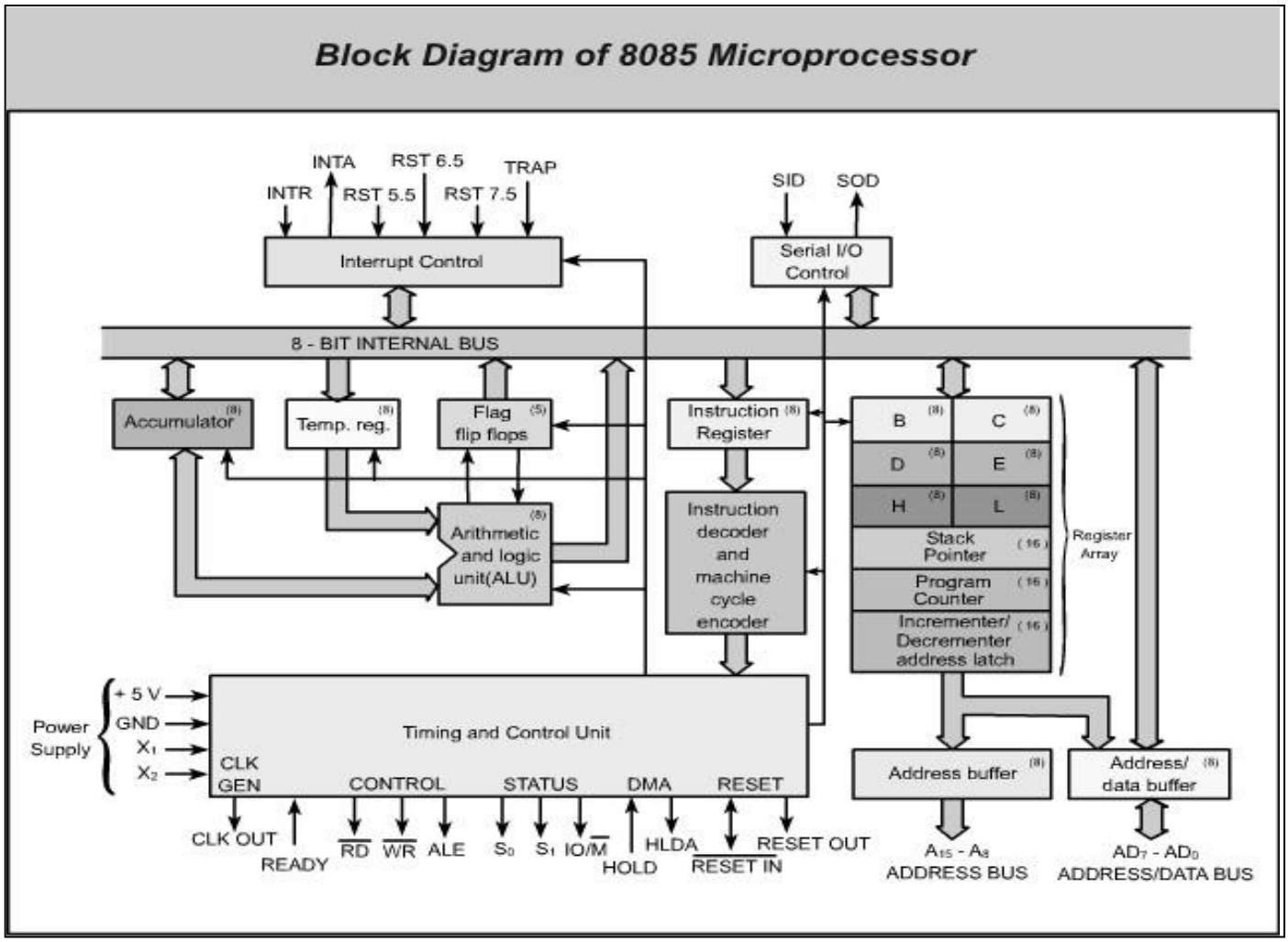
ACCUMULATOR:

- Accumulator is an 8-bit register.
- The accumulator is also called as A-register.
- It holds one of the data to be processed by ALU.
- It stores the result of the operation.
- The accumulator is connected to the 8 – bit internal data bus.
- The two-state output of the accumulator drives the ALU.

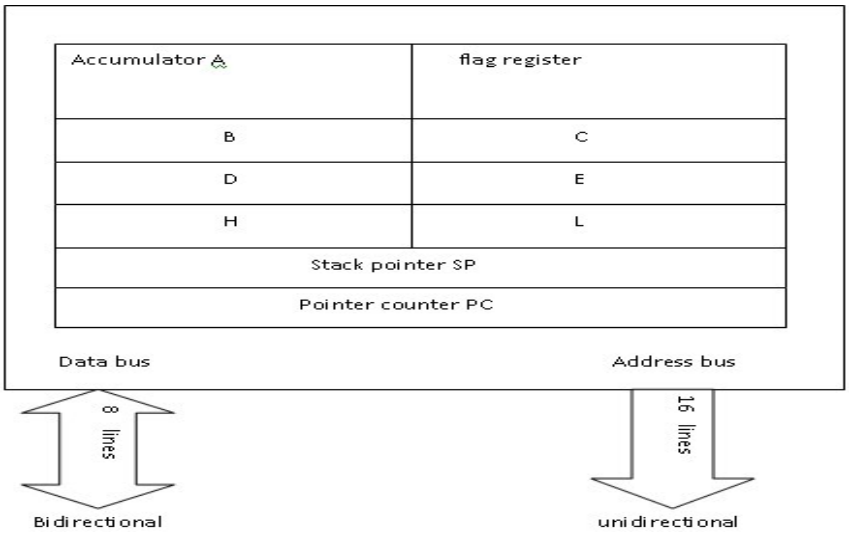
TEMPORARY REGISTER:

- The temporary register receives one of the data to be processed by ALU from external memory or general purpose registers.
- The other input for the ALU comes from the temporary register. This 8-bit register stores the operands of arithmetic – logic operations.

- For instance, during an ADD C the contents of the C register are copied in the temporary register during one T state and added to accumulator contents during another T state.



GENERAL PURPOSE REGISTERS:



- In INTEL 8085 microprocessor, there are six 8 –bit general – purpose registers.
- They are B, C, D, E, H And L.
- They may be used individually or combined as register pairs to perform some 16 – bit operations.
- The permitted combinations of register pairs are B –C, D-E and H-L. The H-L registers pair, which is normally used to form a 16-bit memory pointer.
- The register array (B, C, D, E, H and L) is like a small on-chip RAM with addressable memory locations.
- Control signals select the register for a read or write operation. This means that the CPU can wither load or read a register contents to this data bus.

STACK POINTER (SP):

- Stack Pointer is a 16-bit register used as a memory pointer.
- It maintains the address of the last byte entered into the stack.
- The stack pointer is decremented each time when data is loaded into the stack and is incremented when data is retrieved from the stack.

PROGRAM COUNTER (PC):

- This 16-bit register deals with sequencing the execution of instruction.
- The register is also a memory pointer.
- Memory locations have 16-bit address, and hence it is a 16-bit register.
- The microprocessor uses this register to sequence the execution of the instructions.
- The function of the program counter is to point to the address of the next instruction to be executed.
- At the end of the execution of an instruction, the program counter is incremented by 1, pointing to the next memory location where the next instruction is available.

INCREMENTER/DECREMENTER:

- It can add 1 or subtract 1 from the contents of the Stack Pointer or Program Counter.

ALU:

- The ALU carries out the arithmetic and logic operations on 8-bit words.
- The contents of the accumulator and the temporary register are the inputs to the ALU.
- It can perform arithmetic operations such as addition, subtraction and logical operations such as AND, OR and EX-OR. The ALU result is then stored back in the accumulator.

FLAGS:

Flag register is a group of five individual flip-flops. The content of the flag register will change (0 or 1) after the execution of arithmetic and logic operations.

1. The carry flag bit (CY) is set if a carry or borrow occurs during the arithmetic operation. The carry flag indicates that the operation resulted in overflow.

2. The parity flag bit (P) is set if the result has an even number of 1s, otherwise it will be reset (made zero).

3. The sign flag bit (S) is set if the bit D7 of the result is 1, otherwise it is reset. The sign bit indicates the sign of the number (Positive or Negative) and becomes useful in the signed binary number system.

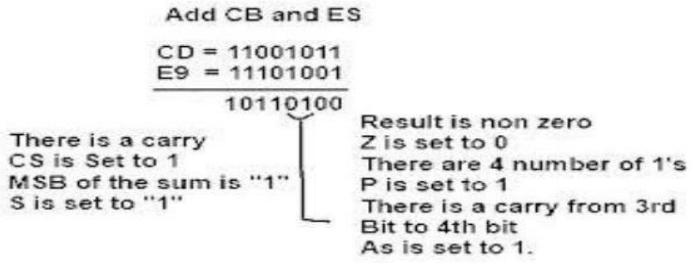
4. The zero flag bit (Z) is set if the result of the operation becomes 0. For all other values of the result the bit is reset.

5. The auxiliary carry flag bit (AC) is set, when a carry is generated at digit D3 position, and passed on to digit D4. The flag is used only internally for Binary Coded Decimal(BCD) operations.

The bit position of different flag register is shown in table.

D7	D6	D5	D4	D3	D2	D1	D0
S	Z	-	AC	P	P	-	CY

Example:



INSTRUCTION REGISTER AND DECODER

- Instruction register and decoder is an 8-bit register.
- When an instruction is fetched from memory, it is loaded in the instruction register.
- The instruction decoder decodes the contents of the instruction register.
- It also determines the operation to be followed in executing the entire instruction and directs the timing and control unit accordingly.
- During the fetch cycle, the opcode of an instruction is stored in the instruction register. This opcode then drives the instruction decoder and machine- cycle encoder.

TIMING AND CONTROL

- The timing and control section of microprocessor includes an oscillator and controller-sequencer.
- The oscillator generates the two – phase clock signals (CLK and) that synchronize all registers. CLK
- The controller-sequencer also produces the control signals needed for internal and external control.

- The controller – sequencer is micro programmed; it has a ROM that stores all the micro routines needed for executing the instruction.
- After each instruction is fetched and stored in the instruction register, the opcode is decoded to get the starting address of the desired micro routine.
- As each microinstruction is read out of the control ROM, control signals are sent to the internal and external data buses.
- The effect is to move data between registers, to perform arithmetic and logic operations, to input or output data, etc.
- The control ROM is sometimes called the control store.

INTERRUPT CONTROL

- Sometimes it is necessary to interrupt the execution of the main program to answer a request from an I/O device.
- For instance, an I/O device may send an interrupt signal to the interrupt control unit to indicate that data is ready for input.
- The computer temporarily stops the execution of main program, inputs the data, and then returns to the main program.
- The interrupt concept is analogous to reading a book (main program), hearing the phone (interrupt), answering the phone (servicing the interrupt), then returning back to reading (main program).

SERIAL I/O CONTROL

Sometimes, I/O devices work with serial data rather than parallel. In this case, the serial data stream from an input device must convert to 8-bit parallel data before the computer can use it. Likewise, the 8-bit data out of a computer mus t be converted to serial form before a serial output device can use it.

The Serial Input Data enters 8085 through pin 5 (SID – Serial Input Data) and leaves through pin 4 (SOD – Serial Input Data) and leaves through pin 4 (SOD – Serial Output Data). Two new instructions known as SIM and RIM allows us to perform the serial- parallel conversion needed for serial I/O devices.

ADDRESS, DATA, AND CONTROL BUSES

Near the top of the Figure is an 8-bit internal data bus. This carries instructions and data between the CPU registers. The external buses are the ones we have to connect to otherchips like memory I/O and so forth. Near the bottom left of the Figure is the external control bus (, ALE). On the bottom right are the external address- data buses. WR RD,

The upper 8 address bits are on a separate bus always used for address bits; this upper section of the address bus is designated A15 - A8. The lower 8-bit are multiplexed. This means that the eight lower bus lines are used for address bits during some T states and for data bits during other T states. This is why the bus is labeled address-data bus, and designed as AD7 – AD0.

ADDRESS BUFFER AND ADDRESS DATA-BUFFER

At the bottom right in Figure are two buffer registers called the address buffer and the address – data buffer. The contents of the stack pointer or program counter can be loaded into the address buffer and address – data buffer. The put of these buffers then drives the external address bus and address –data bus. Memory and I/O chips are connected to these buses. In this way, the CPU can send the address of desired data to the memory or I/O chips.

The 8-bit internal data bus is also connected to the address- data buffer. The bi-directional arrow indicates a three-state connection that allows the address-data buffer to send or receive data from the 8-bit internal data bus.

Features Of 8085

1. 8085A is an 8-bit general – purpose microprocessor.

2. It is a 40 pin dual - in – line package single chip integrated circuit.

3. Only one +5v power supply is needed for its operation.

4. It can operate with a 3 MHZ single – phase clock.

5. The 8085A – 2 versions can operate at the maximum frequency of 5MHZ.

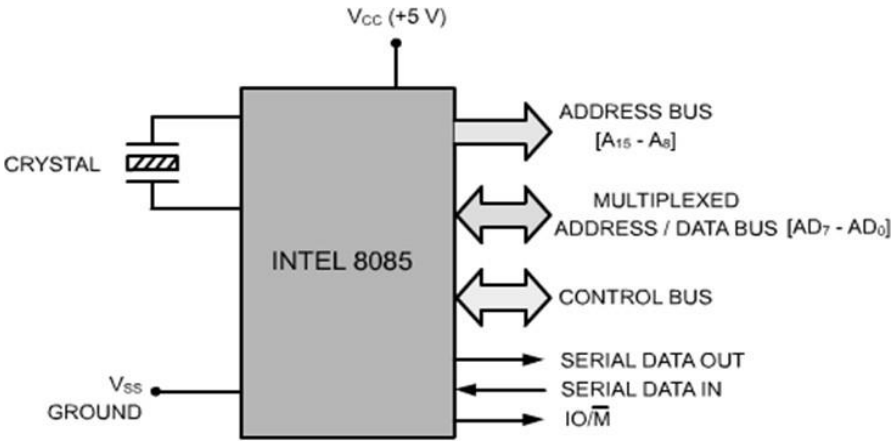
6. The width of the data bus is 8-bit. The width of the address bus is 16 –bit. Therefore maximum of 64 kilobytes of memory locations (i.e. 216=65,536=64KB) can be addressed directly by the 8085.

7. The multiplexing of address/data bus allows for extra control signals.

8. 8085 has one non- maskable (TRAP) and three maskable – vectored interrupts (RST 7.5, 6.5 & 5.5).

9. It provides Serial Input Data (SID) and Serial Output Data(SOD) lines for simple serial interface.

10. 8085 has an inbuilt clock oscillator circuit and requires externally only a crystal. The frequency of the crystal is internally divided by 2.



Bus Organization of INTEL 8085

4. PIN DIAGRAM AND PIN DESCRIPTION OF 8085

- 8085 is a 40 pin IC, DIP.
- The microprocessor is a clock-driven semiconductor device consisting of electronic logic circuits manufactured by using either a large-scale integration (LSI) or very-large-scale integration (VLSI) technique.
- The microprocessor is capable of performing various computing functions and making decisions to change the sequence of program execution.
- In large computers, a CPU implemented on one or more circuit boards performs these computing functions.
- The microprocessor is in many ways similar to the CPU, but includes the logic circuitry, including the control unit, on one chip.
- The microprocessor can be divided into three segments for the sake clarity, arithmetic/logic unit (ALU), register array, and control unit.

POWER SUPPLY AND CLOCK FREQUENCY SIGNALS:

- Vcc + 5 volt power supply
- Vss Ground
- X1, X2 : Crystal or R/C network or LC network connections to set the frequency of internal clock generator.
- The frequency is internally divided by two. Since the basic operating timing frequency is 3 MHz, a 6 MHz crystal is connected externally.
- CLK (output)-Clock Output is used as the system clock for peripheral and devices interfaced with the microprocessor.

The signals from the pins can be grouped as follows:

1. Power supply and clock signals

2. Address bus

3. Data bus

4. Control and status signals

5. Interrupts and externally initiated signals

6. Serial I/O ports

ADDRESS BUS

- A8 - A15 (output; 3-state)
- It carries the most significant 8 bits of the memory address or the 8 bits of the I/O address;

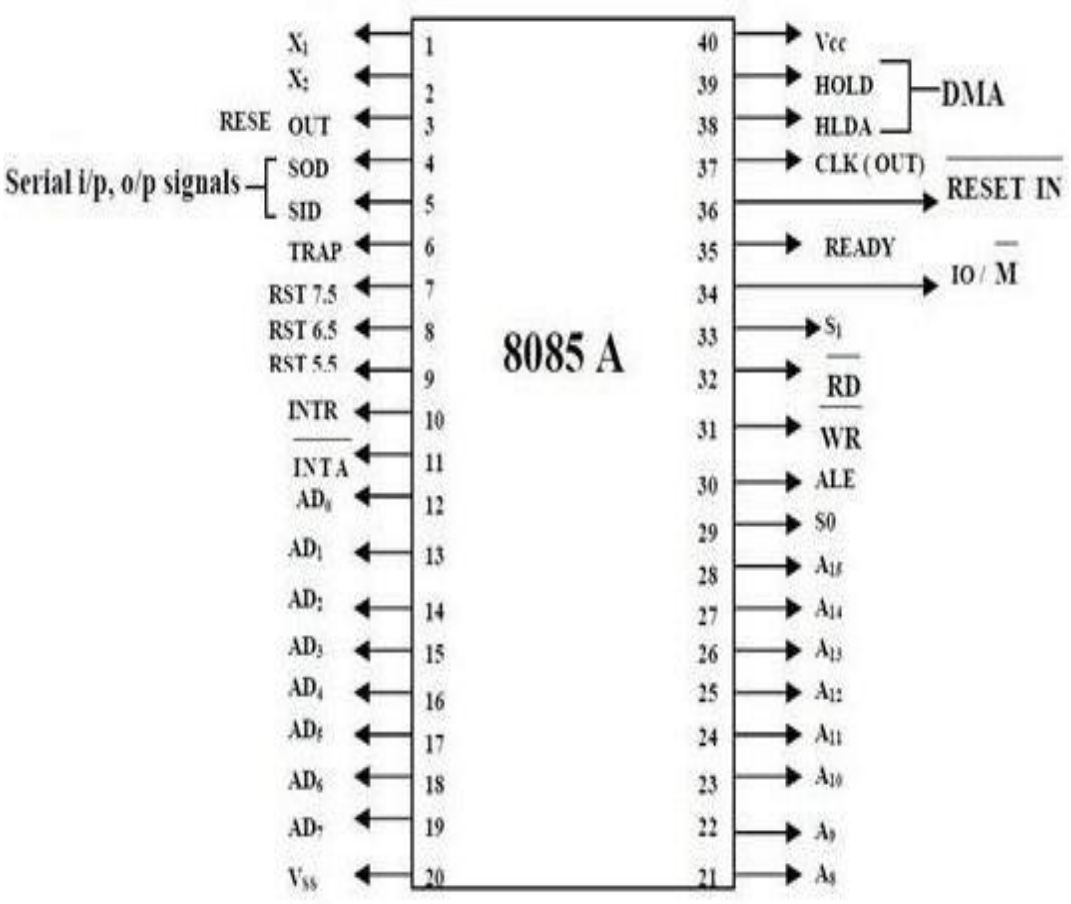


Fig - Pin Diagram of 8085

MULTIPLEXED ADDRESS / DATA BUS

- AD0 - AD7 (input/output; 3-state)
- These multiplexed set of lines used to carry the lower order 8 bit address as well as data bus.

- During the opcode fetch operation, in the first clock cycle, the lines deliver the lower order address A0 - A7.
- In the subsequent IO / memory, read / write clock cycle the lines are used as data bus.

The CPU may read or write out data through these lines

CONTROL AND STATUS SIGNALS

- ALE (output) - Address Latch Enable.
- This signal helps to capture the lower order address presented on the multiplexed address / data bus.
- RD (output 3-state, active low) - Read memory or IO device.
- This indicates that the selected memory location or I/O device is to be read and that the data bus is ready for accepting data from the memory or I/O device.
- WR (output 3-state, active low) - Write memory or IO device.
- This indicates that the data on the data bus is to be written into the selected memory location or I/O device.
- IO/M (output) - Select memory or an IO device.
- This status signal indicates that the read / write operation relates to whether the memory or I/O device.
- It goes high to indicate an I/O operation.
- It goes low for memory operations.

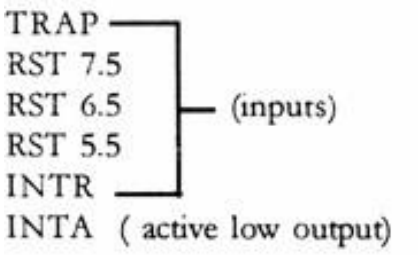
STATUS SIGNALS

- It is used to know the type of current operation of the microprocessor.

IO/M (Active Low)	S1	S2	Data Bus Status (Output)
0	0	0	Halt
0	0	1	Memory WRITE
0	1	0	Memory READ
1	0	1	IO WRITE
1	1	0	IO READ
0	1	1	Opcode fetch
1	1	1	Interrupt acknowledge

INTERRUPTS AND EXTERNALLY INITIATED OPERATIONS

- They are the signals initiated by an external device to request the microprocessor to do a particular task or work.
- There are five hardware interrupts called,



- On receipt of an interrupt, the microprocessor acknowledges the interrupt by the active low INTA (Interrupt Acknowledge) signal.

Reset In (input, active low)

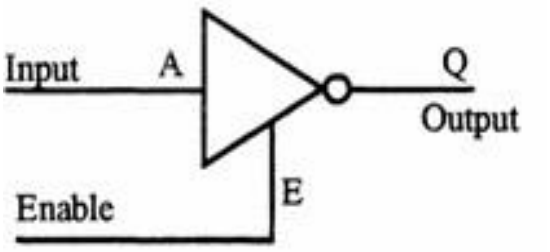
- This signal is used to reset the microprocessor.
- The program counter inside the microprocessor is set to zero.
- The buses are tri-stated.

Reset Out (Output)

- It indicates CPU is being reset.
- Used to reset all the connected devices when the microprocessor is reset.

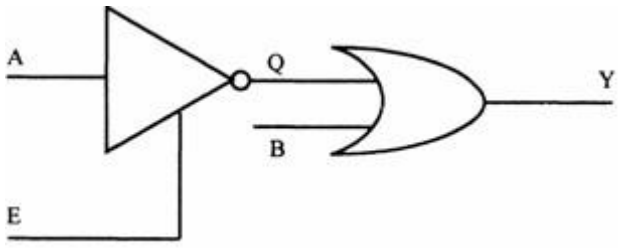
5. DIRECT MEMORY ACCESS (DMA)

Tri state devices:



- 3 output states are high & low states and additionally a high impedance state.
- When enable E is high the gate is enabled and the output Q can be 1 or 0 (if A is 0, Q is 1, otherwise Q is 0). However, when E is low the gate is disabled and the output Q enters into a high impedance state.

E	A	Q	State
1(high)	0	1	High
1	1	0	Low
0(low)	0	0	High impedance
0	1	0	High impedance



- For both high and low states, the output Q draws a current from the input of the OR gate.
- When E is low, Q enters a high impedance state; high impedance means it is electrically isolated from the OR gate's input, though it is physically connected. Therefore, it does not draw any current from the OR gate's input.
- When 2 or more devices are connected to a common bus, to prevent the devices from interfering with each other, the tri state gates are used to disconnect all devices except the one that is communicating at a given instant.
- The CPU controls the data transfer operation between memory and I/O device. Direct Memory Access operation is used for large volume data transfer between memory and an I/O device directly.
- The CPU is disabled by tri-stating its buses and the transfer is effected directly by external control circuits.
- HOLD signal is generated by the DMA controller circuit. On receipt of this signal, the microprocessor acknowledges the request by sending out HLDA signal and leaves out the control of the buses. After the HLDA signal the DMA controller starts the direct transfer of data.

READY (input)

- Memory and I/O devices will have slower response compared to microprocessors.
- Before completing the present job such a slow peripheral may not be able to handle further data or control signal from CPU.
- The processor sets the READY signal after completing the present job to access the data.
- The microprocessor enters into WAIT state while the READY pin is disabled.

Single Bit Serial I/O Ports

- SID (input) - Serial input data line
- SOD (output) - Serial output data line
- These signals are used for serial communication.

6. INSTRUCTION WORD SIZE

The 8085 instruction set is classified into the following three groups according to word size:

1. One-word or 1-byte instructions
2. Two-word or 2-byte instructions
3. Three-word or 3-byte instructions

In the 8085, "byte" and "word" are synonymous because it is an 8-bit microprocessor. However, instructions are commonly referred to in terms of bytes rather than words.

One-Byte Instructions

A 1-byte instruction includes the opcode and operand in the same byte. Operand(s) are internal register and are coded into the instruction
For example:

Task	Opcode	Operand	Binary Code	Hex Code
Copy the contents of the accumulator in the register C.	MOV	C,A	0100 1111	4FH
Add the contents of register B to the contents of the accumulator.	ADD	B	1000 0000	80H
Invert (compliment) each bit in the accumulator.	CMA		0010 1111	2FH

These instructions are 1-byte instructions performing three different tasks. In the first instruction, both operand registers are specified. In the second instruction, the operand B is specified and the accumulator is assumed. Similarly, in the third instruction, the accumulator is assumed to be the implicit operand. These instructions are stored in 8- bit binary format in memory; each requires one memory location.

MOV rd, rs
rd <-- rs copies contents of rs into rd.
Coded as 01 ddd sss where ddd is a code for one of the 7 general registers which is the destination of the data, sss is the code of the source register.

Example: MOV A,B
Coded as 01111000 = 78H = 170 octal (octal was used extensively in instruction design of such processors).

ADD r
A <-- A + r

Two-Byte Instructions

In a two-byte instruction, the first byte specifies the operation code and the second byte specifies the operand. Source operand is a data byte immediately following theopcode.
For example

Task	Opcode	Operand	Binary Code	Hex Code	
Load an 8-bit data byte in the accumulator.	MVI	A, Data	0011 1110 DATA	3E Data	First Byte Second Byte

Assume that the data byte is 32H. The assembly language instruction is written as

Mnemonics	Hex code
MVI A, 32H	3E 32H

The instruction would require two memory locations to store in memory.

MVI r,data
r <-- data
Example: MVI A,30H coded as 3EH 30H as two contiguous bytes. This is an example of immediate addressing.
ADI data
A <-- A + data
OUT port

where port is an 8-bit device address. (Port) <-- A. Since the byte is not the data but points directly to where it is located this is called direct addressing.

Three-Byte Instructions

In a three-byte instruction, the first byte specifies the opcode, and the following two bytes specify the 16-bit address. Note that the second byte is the low-order address and the third byte is the high-order address.
opcode + data byte + data byte

For example:

Task	Opcode	Operand	Binary code	Hex Code	
Transfer the program sequence to the memory location 2085H.	JMP	2085H	1100 0011 1000 0101 0010 0000	C3 85 20	First byte Second Byte Third Byte

This instruction would require three memory locations to store in memory.

Three byte instructions - opcode + data byte + data byte

LXI rp, data16

rp is one of the pairs of registers BC, DE, HL used as 16-bit registers. The two data bytes are 16-bit data in L H order of significance.
rp <-- data16

Example:
LXI H,0520H coded as 21H 20H 50H in three bytes. This is also immediate addressing.

LDA addr

A <-- (addr) Addr is a 16-bit address in L H order. Example: LDA 2134H coded as 3AH 34H 21H. This is also an example of direct addressing.

7. ADDRESSING MODES OF 8085

- Every instruction of a program has to operate on a data.
- The method of specifying the data to be operated by the instruction is called Addressing**.**
- The various ways of specifying the operand in the operand field of an instruction are called the addressing modes.

The various addressing modes are:

1. Direct addressing mode
2. Immediate addressing mode
3. Register direct addressing mode
4. Register indirect addressing mode
5. Implicit addressing mode
6. Stack addressing mode
7. Indirect addressing mode
8. Indexed addressing mode
9. Relative addressing mode

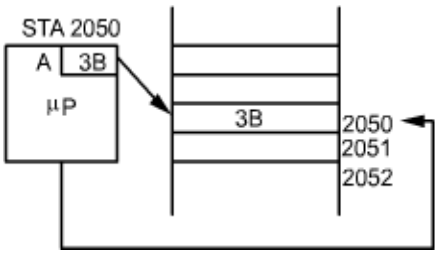
1) DIRECT ADDRESSING MODE:

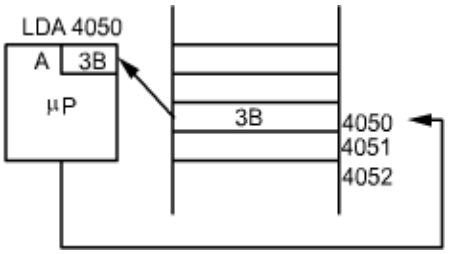
In direct addressing mode, the address of the operand is directly specified in the instruction. Except IN and OUT instructions all other direct addressing modes are 3-bytes long.

Examples:

a) **STA 16-bit Address** – The contents of the accumulator are copied to a memory location whose address is specified.

b) LDA 16-bit Address – The contents of the memory location whose address is specified in byte 2 and byte 3 of the instructions are copied to the Accumulator. It is 3-byte instruction.





2) IMMEDIATE ADDRESSING MODE:

In immediate addressing mode, the actual data (8-bit or 16-bit) is part of the instruction. The length of the instruction may be two or three bytes. The first byte specifies opcode. If it is a two byte instruction, the second byte specifies an 8-bit data. If it is a three byte instruction, the second and third bytes specify 16-bit data.

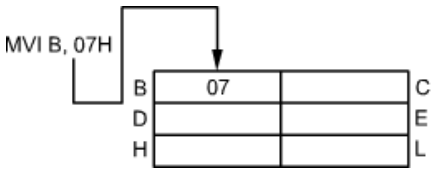
Examples:

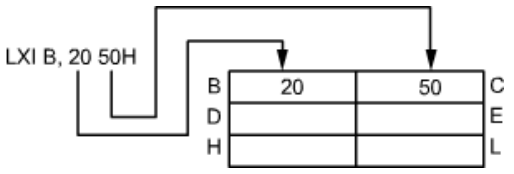
a) MVI R, 8-bit Data

It is a 2 byte instruction. Byte 2 (8-bit data) of the instruction is immediately moved to register R (R may be A, B, C, D, E, H, L).

b) LXI R_P, 16-bit Data

It is a 3-byte instruction. Byte 2 of the instruction is immediately moved into the low-order register of the register pair R_p and byte 3 of the instruction immediately moved into the high-order register of the register pair.





3) REGISTER DIRECT ADDRESSING MODE:

In some instructions, general-purpose registers are specified as the address of the operands. Such instructions are called as register direct addressing mode instructions. These instructions are one byte long. Since the microprocessor need not fetch data from memory, this mode of addressing is faster than direct addressing mode. This mode of addressing is called as register addressing mode.

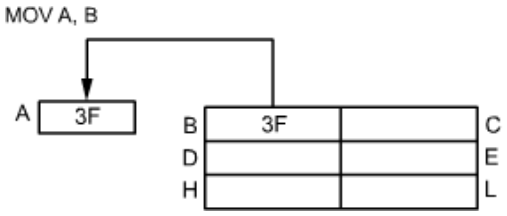
Examples:

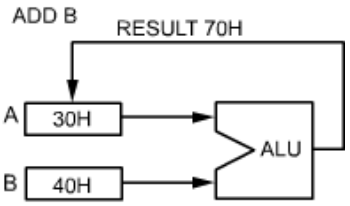
a) MOV R_d, R_s

The contents of the source register (R_s) are moved to the destination register (R_d). It is a single byte instruction (R_d and R_s are general purpose registers).

b) ADD R

It is a single byte instruction. The contents of the register pair 'R_p'. (R_p may be BC, DE, HL) are incremented by 1.





4) REGISTER INDIRECT ADDRESSING MODE:

In register indirect addressing mode the contents of the specified register pair is used as the address of the operand. The register pair contains the 16-bit address of the memory location where the actual operand is stored. Usually the memory pointer (HL-register pair) contains the address of the memory location.

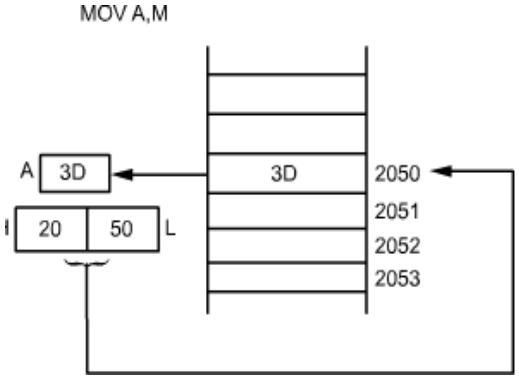
Examples

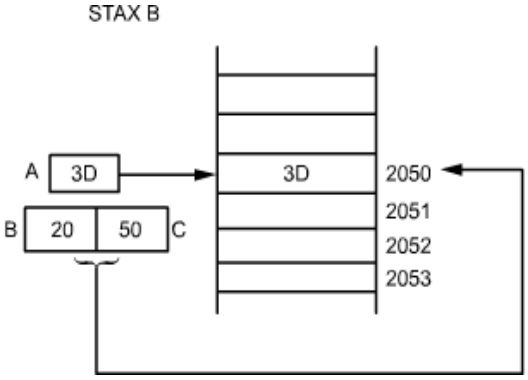
a) MOV R_d ,M

It is a single byte instruction. The contents of the memory location whose address is specified by the contents of the HL-register pair are moved to the destination register R_d (R_d may be any one of the general purpose register).

b) ADD M

It is a single byte instruction. The contents of the memory location whose address is specifies by the contents of the memory pointer (HL – register pair) are added with the contents of the accumulator and the result is placed in the accumulator.





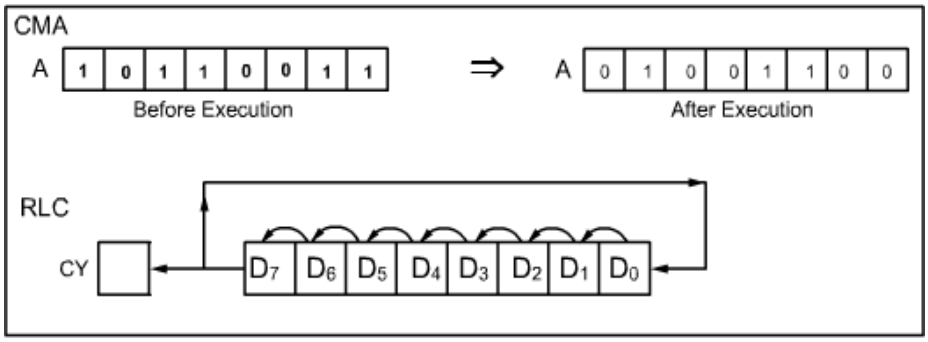
5) IMPLICIT ADDRESSING MODE

In implicit addressing mode, the address of a register (Accumulator in the case of 8085 containing the operand data) is implicitly stated in the opcode itself. In this addressing mode, the instructions are one byte long since the operand is in the accumulator.

Examples

a) CMA – It is a single byte instruction. The content of the accumulator is completed. There the operand (data) which is nothing but the contents of accumulator is specified within the instruction.

b) RLC – It is a single byte instruction. The contents of the accumulator are rotated left by one position.

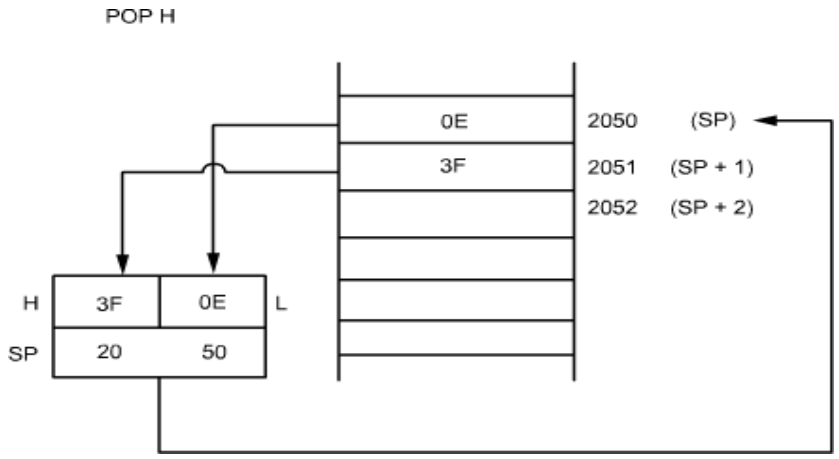


6) STACK ADDRESSING MODE

In this addressing the content of the stack pointer (16-bit register) is the address of the operand (data). It is similar to register addressing mode. Here the stack pointer content is the address of the stack memory.

Examples

POP R_p – It is a single byte instruction. The contents of the memory location pointed by the stack pointer are copied to the low-order-register. The stack pointer is incremented by 1 and contents of that memory location are copied to the high order register.

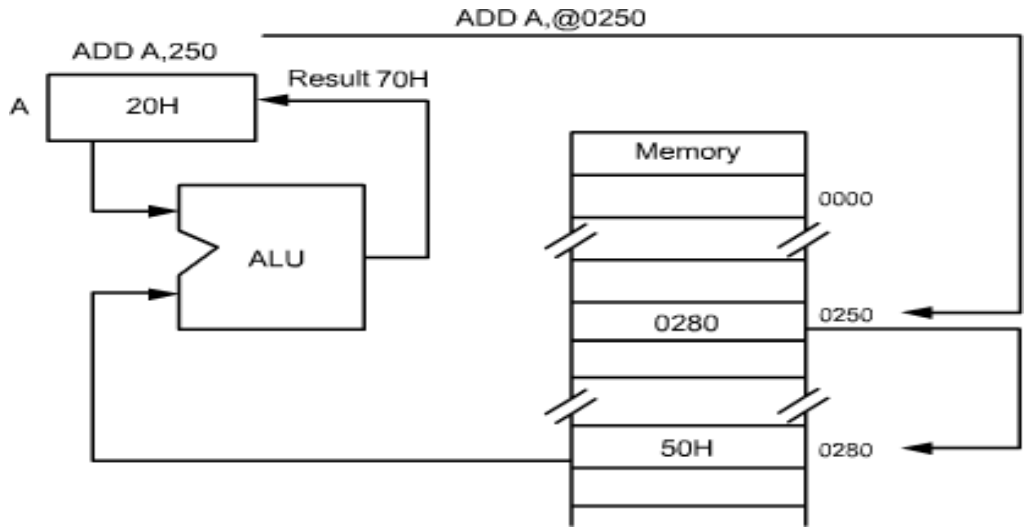


7. INDIRECT ADDRESSING MODE:

In Indirect addressing mode, the instruction points to an Address where the exact address of the operand is present.

Examples

ADD A, 2050 – In this instruction the contents of the accumulator are added with the content of memory location, whose Address is specified by the operand of the instruction.



8. INDEXED ADDRESSING MODE

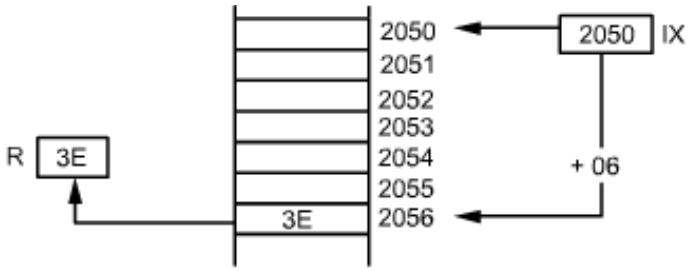
In this Addressing mode the Address of the operand is specified in relation to the contents of a 16-bit register called "Index Register". A displacement, which is to be added with the contents of the index, register is also given in the instruction itself. This type of Addressing mode is used in Z-80 microprocessor.

Examples

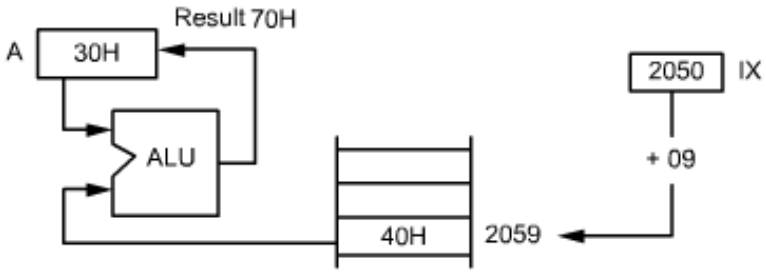
a) **LD R, (IX + d) –** This instruction will move the contents of the memory location specified by (IX + d) into specified register.

b) **ADD A, (IX +d) –** This instruction will add the contents of the memory location specified by (IX + d) with the contents of the accumulator.

LD R, (IX+d)
Let IX = 2050
d = 06
∴ **LD R, (2050+06)**
LD R, 2056



ADD A, (IX+d)
Let IX = 2050
d = 09
∴ **ADD A, (2050+09)**
ADD A,2059



8. RELATIVE ADDRESSING MODE

In this type of addressing mode, in order to find the effective address the address specified in the instruction (referred as offset) is added with the contents of PC. This mode of addressing is used in Motorola 6800 and Z – 80 microprocessor.

Offset is the displacement of the branching location from the Branch instruction.

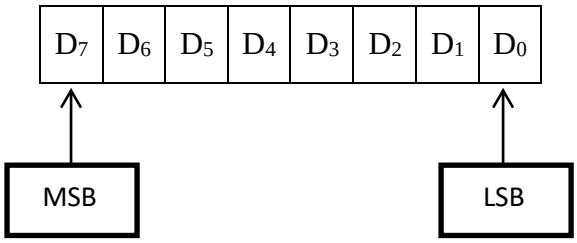
Offset = address of the loop – address of next instruction to the branch instruction

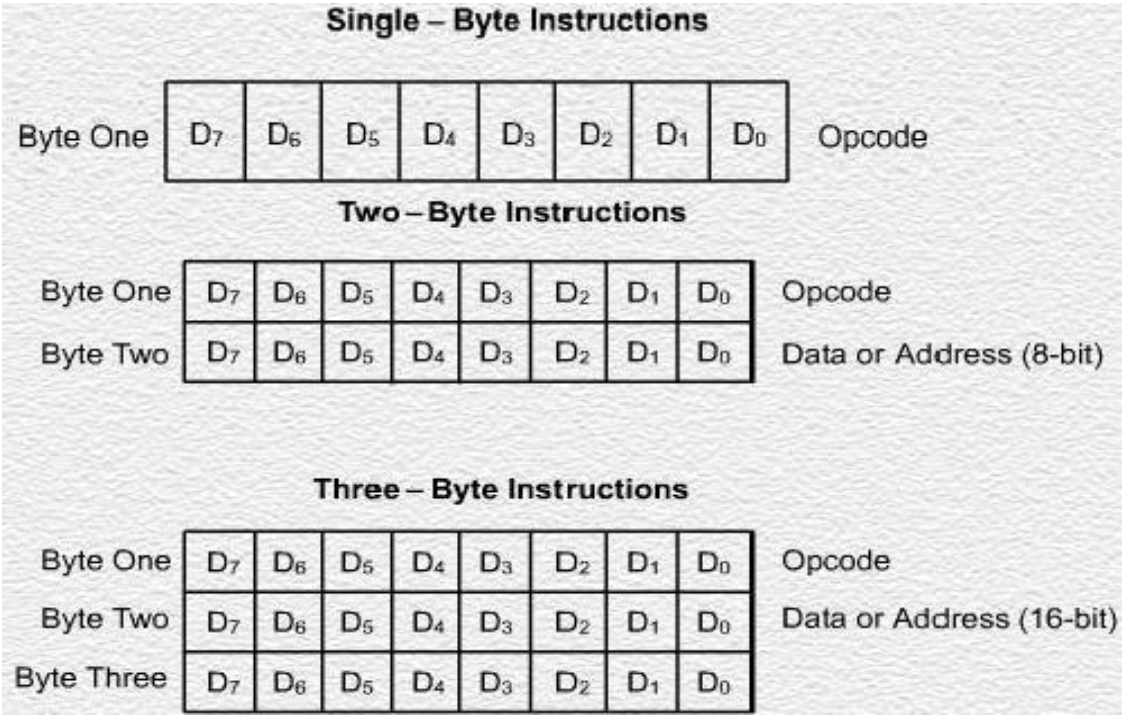
8. INSTRUCTION SET OF 8085

The 8085A implements a group of instructions that move data between registers, between a register and memory, and between registers and an I/O Port. It also has arithmetic and logic instructions, conditional branch instructions. The CPU recognizes these instructions only when they are coded in binary form.

Instruction and Data Formats

Data in the 8085A is stored in the form of 8-bit values.





The complete 8085 instruction set is described, grouped under five different functional headings, as follows

1. DATA TRANSFER INSTRUCTIONS

It includes the instruction that moves (copies) data between memory location and register. In all data transfer operations the content of source register / memory is not altered. Hence the data transfer is copying instruction.

Opcode	Operand	Description

Copy from source to destination

MOV	**R_d, R_s**	This instruction copies the contents of the source register into the destination register; the contents of the source register are not altered. If one of the operands is a memory location, its location is specified by the contents of the HL registers. **Example:** MOV B, C or MOV B, M
	M, R_s	
	R_d, M	

Load accumulator

LDA	**16-bit address**	The contents of a memory location, specified by a 16-bit address in the operand, are copied to the accumulator. The contents of the source are not altered. **Example:** LDA 2034H

Store accumulator direct

STA	**16-bit address**	The contents of the accumulator are copied into the memory location specified by the operand. This is a 3-byte instruction, the second byte specifies the low-order address and the third byte specifies the high-order address. **Example:** STA 4350H

Exchange H and L with D and E

XCHG	**none**	The contents of register H are exchanged with the contents of register D, and the contents of register L are exchanged with the contents of register E. **Example:** XCHG

Push register pair onto stack

PUS	**Reg. pair**	The contents of the register pair designated in the operand are copied onto the stack in the following sequence. The stack pointer register is decremented and the contents of the high-order register (B, D, H, A) are copied into that location. The stack pointer register is decremented again and the contents of the low-order register (C, E, L, flags) are copied to that location. **Example:** PUSH B or PUSH A

Pop off stack to register pair

POP	**Reg. pair**	The contents of the memory location pointed out by the stack pointer register are copied to the low-order register (C, E, L, status flags) of the operand. The stack pointer is incremented by 1 and the contents of that memory location are copied to the high-order register (B, D, H, A) of the operand. The stack pointer register is again incremented by 1. **Example:** POP H or POP A

Output data from accumulator to a port with 8-bit address

OUT	**8-bit port address**	The contents of the accumulator are copied into the I/O port specified by the operand. Example: OUT F8H

2. ARITHMETIC INSTRUCTIONS

It includes the instruction which performs addition, subtraction, increment, decrement operations. The flag conditions are altered after execution of an instruction in this group.

Opcode Operand Description

Add register or memory to accumulator

ADD	**R**	The contents of the operand (register or memory) are added to the contents of the accumulator and the result is stored in the accumulator. If the operand is a memory location, its location is specified by the contents of the HL registers. All flags are modified to reflect the result of the addition. **Example:** ADD B or ADD M
ADD	**M**	

Subtract immediate from accumulator

SUI	**8-bit data**	The 8-bit data (operand) is subtracted from the contents of the accumulator and the result is stored in the accumulator. All flags are modified to reflect the result of the subtraction. **Example:** SUI 45H

Decimal adjust accumulator

DAA	**none**	The contents of the accumulator are changed from a binary value to two 4-bit binary coded decimal (BCD) digits. This is the only instruction that uses the auxiliary flag to perform the binary to BCD conversion, and the conversion procedure is described below. S, Z, AC, P, CY flags are altered to reflect the results of the operation.
		If the value of the low-order 4-bits in the accumulator is greater than 9 or if AC flag is set, the instruction adds 6 to the low-order four bits.
		If the value of the high-order 4-bits in the accumulator is greater than 9 or if the Carry flag is set, the instruction adds6 to the high-order four bits.
		Example: DAA

Increment register pair by 1

INX	**R**	The contents of the designated register pair are incremented by 1 and the result is stored in the same place. **Example:** INX H

3. BRANCHING INSTRUCTIONS

The instructions which performs the logical operations like AND, OR, EX-OR, complement, compare and rotate instructions are grouped under this heading. The flag conditions are altered after the execution of an instruction in this group.

Opcode Operand Description

Jump unconditionally

JMP	**R**	The program sequence is transferred to the memory location specified by the 16-bit address given in the operand. **Example:** JMP 2034H or JMP XYZ

Opcode	**Description**	**Flag Status**
JC	Jump on Carry	CY = 1
JNC	Jump on no Carry	CY = 0
JP	Jump on positive	S = 0
JM	Jump on minus	S = 1
JZ	Jump on zero	Z = 1
JNZ	Jump on no zero	Z = 0
JPE	Jump on parity even	P = 1
JPO	Jump on parity odd	P = 0

Unconditional subroutine call

CALL	**16-bit address**	The program sequence is transferred to the memory location specified by the 16-bit address given in the operand. Before the transfer, the address of the next instruction after CALL (the contents of the program counter) is pushed onto the stack. Example: CALL 2034H or CALL XYZ

Opcode	**Description**	**Flag Status**
CC	Call on Carry	CY = 1
CNC	Call on no Carry	CY = 0
CP	Call on positive	S = 0
CM	Call on minus	S = 1

CZ	Call on zero	Z = 1
CNZ	Call on no zero	Z = 0
CPE	Call on parity even	P = 1
CPO	Call on parity odd	P = 0

4. LOGICAL INSTRUCTIONS

The instructions that are used to transfer the program control from one memory location to another memory location are grouped under this heading.

Opcode Operand Description

Compare register or memory with accumulator

CMP	**R**	The contents of the operand (register or memory) are compared with the contents of the accumulator. Both contents are preserved. The result of the comparison is shown by setting the flags of the PSW as follows:
		if (A) < (reg/mem):
		carry flag is
		set if (A) = (reg/mem):
		zero flag is set
		if (A) > (reg/mem): carry and zero flags are reset
		Example: CMP B or CMP M

	M	

Logical AND register or memory with accumulator

ANA	**R**	The contents of the accumulator are logically ANDed with the contents of the operand (register or memory), and the result is placed in the accumulator. If the operand is a memory location, its address is specified by the contents of HL registers. S, Z, P are modified to reflect the result of the operation. CY is reset. AC is set. **Example:** ANA B or ANA M
	M	

Exclusi0ve OR register or memory with accumulator

XRA	**R**	The contents of the accumulator are Ex-ORed with
	M	the contents of the operand (register or memory), and the
		result is placed in the accumulator. If the operand is a
		memory location, its address is specified by the contents of
		HL registers. S, Z, P are modified to reflect the result of the
		operation. CY and AC are reset.
		Example: XRA B or XRA M

5. MACHINE CONTROL INSTRUCTIONS

It includes the instructions related to interrupts and the instructions used to halt program execution.

Opcode	Operand	Description

Halt and enter wait state

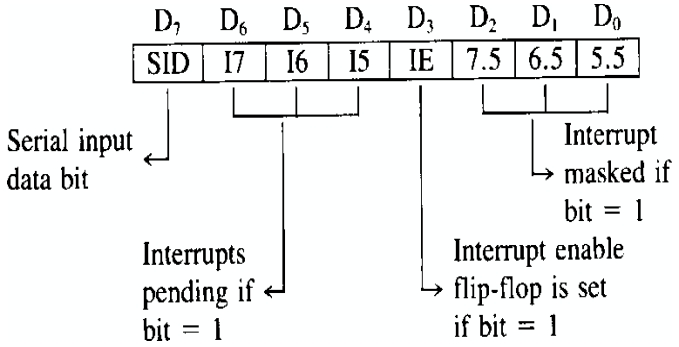
HLT	**none**	The CPU finishes executing the current instruction and halts any further execution. An interrupt or reset is necessary to exit from the halt state. **Example**: HLT

Enable interrupts

EI	**none**	The interrupt enable flip-flop is set and all interrupts are enabled. No flags are affected. After a system reset or the acknowledgement of an interrupt, the interrupt enable flip-flop is reset, thus disabling the interrupts. This instruction is necessary to reenable the interrupts (except TRAP). **Example**: EI

Read interrupt mask

RIM	**none**	This is a multipurpose instruction used to read the status of interrupts 7.5, 6.5, 5.5 and read serial data input bit. The instruction loads eight bits in the accumulator with the following interpretations. **Example:** RIM



9. ASSEMBLY LANGUAGE PROGRAMMING:

Assembler:

It is software that converts assembly language program code to machine language code.

Assembly language instructions have the format:

ADDRESS	LABEL	MNEMONICS	OPERAND	COMMENTS

Address

- Specify the address of an instruction.

Instruction Label (optional)

- Marks the address of an instruction, must have a colon :
- Used to transfer program execution to a labeled instruction

Mnemonic

- Identifies the operation (e.g. MOV, ADD, SUB, JMP, CALL)

Operands

- Specify the data required by the operation
- Executable instructions can have zero to three operands
- Operands can be registers, memory variables, or constants

No operands

```
stc ; set carry flag
```

One operand

```
inc eax ; increment register eax
call Clrscr ; call procedure Clrscr
jmp L1 ; jump to instruction with label L1
```

Two operands

```
add ebx, ecx ; register ebx = ebx + ecx
sub var1, 25 ; memory variable var1 = var1 – 25
```

Three operands

```
imul eax,ebx,5          ; register eax = ebx * 5
```

Identifiers

- Identifier is a programmer chosen name
- Identifies variable, constant, procedure, code label
- May contain between 1 and 247 characters
- Not case sensitive

- First character must be a letter (A..Z, a..z), underscore(_), @, ?, or $.
- Subsequent characters may also be digits.
- Cannot be same as assembler reserved word.

Comments

- Comments are very important!
    - Explain the program's purpose
    - When it was written, revised, and by whom
    - Explain data used in the program
    - Explain instruction sequences and algorithms used
    - Application-specific explanations
- Single-line comments
    - Begin with a semicolon ; and terminate at end of line
- Multi-line comments
    - Begin with COMMENT directive and a chosen character
    - End with the same chosen character

Arithmetic Operation:

Program in C:

```
void main()
{
int a=5,b=6,c;    // define the data type for  variable a,b& cand intialize the value for variable a
& b respectively.
c=a+b;              // add the varible a & b and place the result in C
printf("%d",c); // display the value of variable c
}
```

Program in Microprocessor (immediate addressing):

```
MVI A, 05      // assign the value 05 to accumulator
MVI B, 06      // assign the value 06 to B register
```

ADD B // add the content in B register with accumulator and place the result in accumulator

STA 4200 // store the result to the memory location 4200. HLT // stop the program

Program in C:

```
Void main()
{
Int a, b, c; \\define the data type for variable
Printf("enter the value for a & b");
Scanf("%d%d"&a,&b); // get the number and place it to the memory respectively.
C=a+b; // add the variable a & b and place the result into c.
Printf("reult is %d"c); // display the value of c;
}
```

Program in Microprocessor (direct addressing):

LDA 4500 // load the content of memory location 4500 to accumulator. MOV B,A // move the content from accumulator to B register.

LDA 4501 // load the content of memory location 4501 to accumulator

ADD B // add the content in B register with accumulator and place the result in accumulator

STA 4502 // store the result to the memory location 4200.

HLT // stop the program.

Logic operation:

```
//C program for Arranging 5 Numbers in Ascending Order
#include<stdio.h>
#include<conio.h>
void main()
{
int a[5],i,j,t; // define the variables.
clrscr();
printf("Enter 5 nos.\n\n");
```

```
for (i=0;i<5;i++) // perform loop operation.
scanf("%d",&a[i]); // get the given number to store it in the respective address .
for (i=0;i<5;i++) // perform loop operation.
{ for(j=i+1;j<5;j++) // perform inner loop operation.
{ if(a[i]>a[j]) // compare the numbers a[i] & a[j] respectively
{ t=a[i]; // use the temporary variable for making swap operation.
a[i]=a[j];
a[j]=t;
} } }
printf("Ascending Order is:");
for(j=0;j<5;j++) // perform the loop operation for display the number in ascending order.
printf("\n%d",a[j]);
getch();
}
```

ASCENDING ORDER

Aim:

To write a program to sort given 'n' numbers in ascending order

Algorithm:

1. Load the count value from memory to A-reg and save it in B-reg.
2. Decrement B-reg (B is a count for (N-1) repetitions)
3. Set HL pair as data address pointer.
4. Set C-reg as counter for(N-1) comparisons.
5. Load a data of the array in accumulator using the data address pointer.
6. Increment the HL pair(data address pointer).
7. Compare the data pointed by HL with accumulator.
8. If Carry flag is set(if the content of accumulator is smaller than memory)then goto step10,otherwise go to next step.
9. Exchange the content of memory pointed by HL and the accumulator.

10. Decrement C-register. If zero flag is reset go to step 6 otherwise go to next step.

11. Decrement B-register. If zero flag is reset go to step 3 otherwise go to next step.

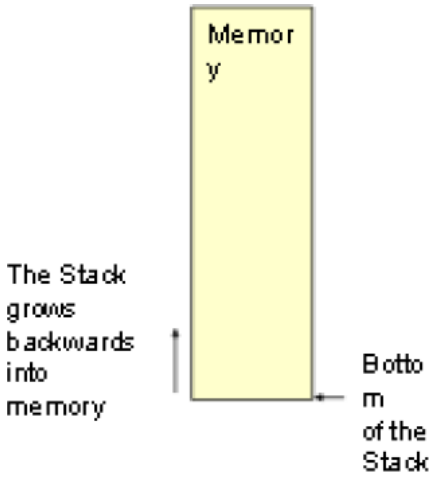
12. Stop.

Program:

ADDRESS	LABEL	MNEMONICS	OPCODE	COMMENTS
4100		LDA 4200	3A,00,42	Load the count value in A-reg.
4103		MOV B, A	47	Set counter for (N-1) repetition of
				(N-1) comparisons.
4104		DCR B	05	
4105	LOOP2:	LXI H, 4200H	21,00,42	Set pointer for array
4108		MOV C, M	4E	Set counter for of (N-1) comparisons.
4109		DCR C	0D	Decrement the content of C-reg.
410A		INX H	23	Increment the HL-reg pair.
410B	LOOP1:	MOV A,M	7E	Move the content of memory pointer to accumulator.
410C		INX H	23	Increment the HL-reg pair.
410D		CMP M	BE	Compare the accumulator with memory.
410E		JC AHEAD	DA,16,41	If the content of A-reg is less than memory address of HL-reg pair then go to AHEAD.
4111		MOV D, M	56	If the content of A-reg is greater than the memory address of HL- reg pair, then exchange the content of memory pointed by HL and previous location.
4112		MOV M, A	77	
4113		DCX H	2B	
4114		MOV M,D	72	
4115		INX H	23	Increment the HL-reg pair.
4116	AHEAD:	DCR C	0D	Decrement the content of C-reg.
4117		JNZ LOOP1	C2,0B,41	Repeat comparisons until C count is zero.
411A		DCR B	05	
411B		JNZ LOOP2	C2,05,41	Repeat until B count is zero.
411E		HLT	76	Stop the program

10.THE STACK

- The stack is an area of memory identified by the programmer for temporary storage of information.
- The stack is a LIFO structure.
  -Last In First Out.
- The stack normally grows backwards into memory.
  -In other words, the programmer defines the bottom of the stack and the stack grows up into reducing address range.



- Given that the stack grows backwards into memory, it is customary to place the bottom of the stack at the end of memory to keep it as far away from user programs as possible.
- In the 8085, the stack is defined by setting the SP (Stack Pointer) register.

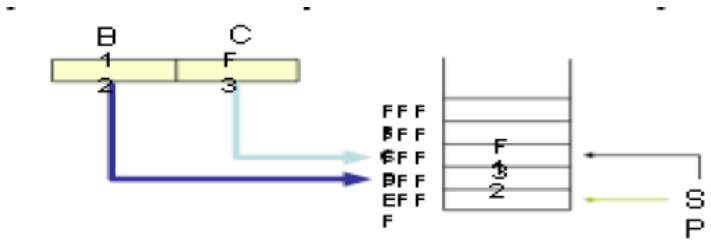
  LXI SP, FFFFH
- This sets the Stack Pointer to location FFFFH (end of memory for the 8085).

Saving Information on the Stack

- Information is saved on the stack by PUSHing it on. It is retrieved from the stack by POPing it off.
- The 8085 provides two instructions:
  - PUSH and POP for storing information on the stack and retrieving it back.
  - Both PUSH and POP work with register pairs ONLY.

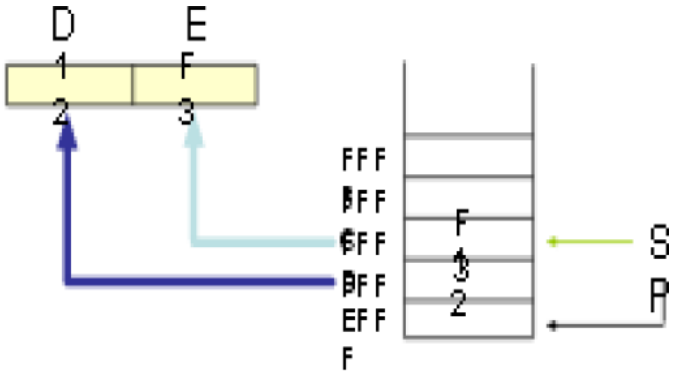
The PUSH Instruction

- PUSH B
  - Decrement SP
  - Copy the contents of register B to the memory location pointed to by SP
  - Decrement SP
  - Copy the contents of register C to the memory location pointed to by SP



The POP Instruction

- POP D
  - Copy the contents of the memory location pointed to by the SP to register E
  - Increment SP
  - Copy the contents of the memory location pointed to by the SP to register D
  - Increment SP



Operation of the Stack

- During pushing, the stack operates in a "decrement then store" style.
  - The stack pointer is decremented first, then the information is placed on the stack.
- During poping, the stack operates in a "use then increment" style.
  - The information is retrieved from the top of the the stack and then the pointer is incremented.
- The SP pointer always points to "the top of the stack".

LIFO

- The order of PUSHs and POPs must be opposite of each other in order to retrieve information back into its original location.

PUSH B
PUSH D
.....
POP D
POP B

The PSW Register Pair

- The 8085 recognizes one additional register pair called the PSW (Program Status Word).
  - This register pair is made up of the Accumulator and the Flags registers.

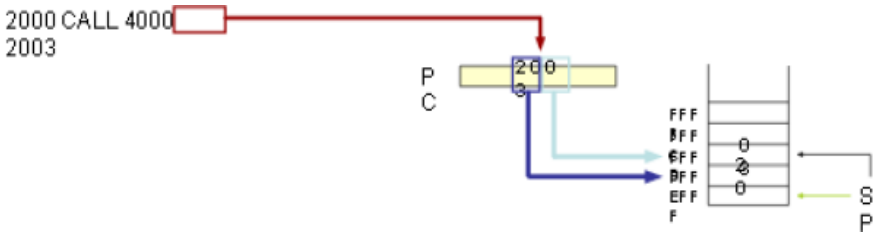
- It is possible to push the PSW onto the stack, do whatever operations are needed, then POP it off of the stack.
    - The result is that the contents of the Accumulator and the status of the Flags are returned to what they were before the operations were executed.

11. SUBROUTINES

- A subroutine is a group of instructions that will be used repeatedly in different locations of the program.
    - Rather than repeat the same instructions several times, they can be grouped into a subroutine that is called from the different locations.
- In Assembly language, a subroutine can exist any where in the code.
    - However, it is customary to place subroutines separately from the main program.
- The 8085 has two instructions for dealing with subroutines.
    - The CALL instruction is used to redirect program execution to the subroutine.
    - The RTE instruction is used to return the execution to the calling routine.

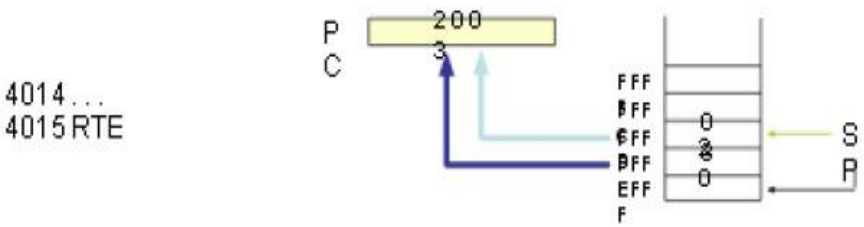
The CALL Instruction

- CALL 4000H
    - Push the address of the instruction immediately following the CALL onto the stack
    - Load the program counter with the 16-bit address supplied with the CALL instruction.



The RTE Instruction

- RTE
    - Retrieve the return address from the top of the stack
    - Load the program counter with the return address.



- The CALL instruction places the return address at the two memory locations immediately before where the Stack Pointer is pointing.
    - You must set the SP correctly BEFORE using the CALL instruction.
- The RTE instruction takes the contents of the two memory locations at the top of the stack and uses these as the return address.
    - Do not modify the stack pointer in a subroutine. You will loose the return address.

Passing Data to a Subroutine

- In Assembly Language data is passed to a subroutine through registers.
    - The data is stored in one of the registers by the calling program and the subroutine uses the value from the register.
- The other possibility is to use agreed upon memory locations.
    - The calling program stores the data in the memory location and the subroutine retrieves the data from the location and uses it.

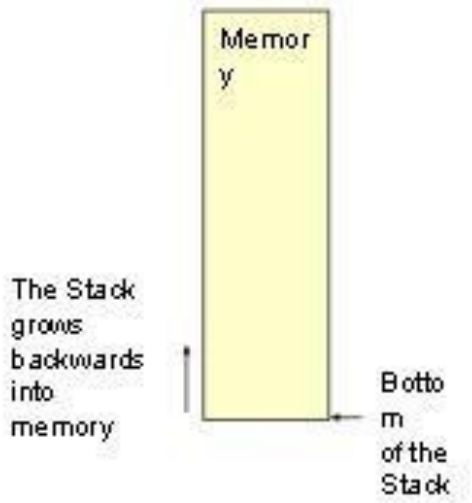
Call by Reference and Call by Value

- If the subroutine performs operations on the contents of the registers, then these modifications will be transferred back to the calling program upon returning from a subroutine.

    - Call by reference.
- If this is not desired, the subroutine should PUSH all the registers it needs on the stack on entry and POP them on return.
    - The original values are restored before execution returns to the calling program.

Cautions with PUSH and POP

- PUSH and POP should be used in opposite order.



re are PUSH's.
l pick up the wrong information from the top of the
l.
POP inside a loop.

, and conditional RTE instructions.
h conditional JUMP instructions can be used.
Carry flag is set.
f Carry flag is not set.
utine if Carry flag is set.
routine if Carry flag is not set Etc.

12. TIMING DIAGRAM

OPCODE FETCH MACHINE CYCLE OF 8085:

- Each instruction of the processor has one byte opcode.
- The opcodes are stored in memory. So, the processor executes the opcode fetch machine cycle to fetch the opcode from memory.
- Hence, every instruction starts with opcode fetch machine cycle.
- The time taken by the processor to execute the opcode fetch cycle is 4T.
- In this time, the first, 3 T-states are used for fetching the opcode from memory and the remaining T-states are used for internal operations by the processor.

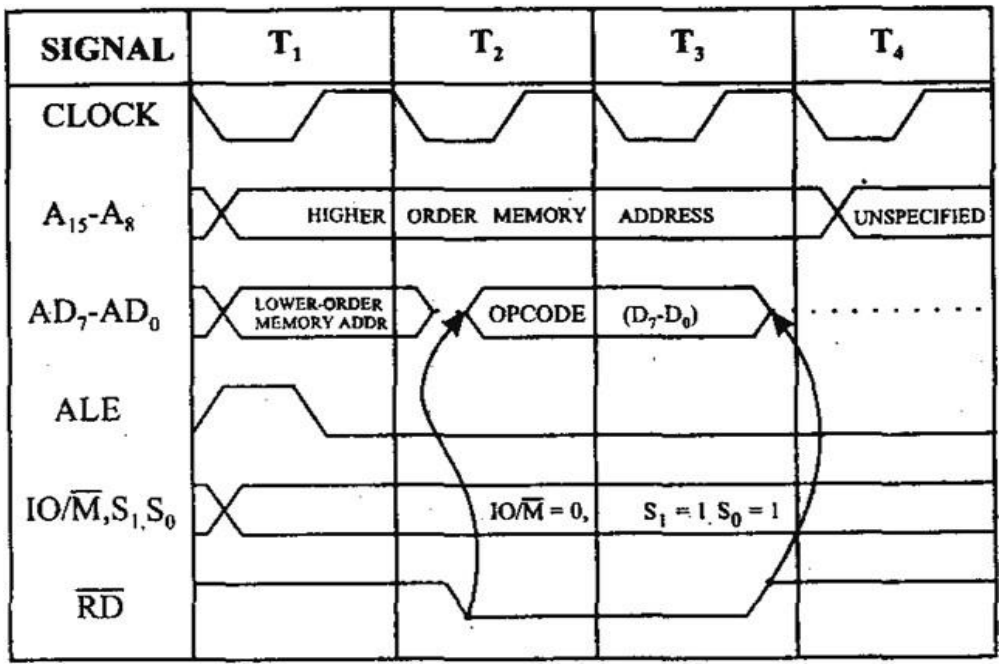


Fig - Timing Diagram for Opcode Fetch Machine Cycle

MEMORY READ MACHINE CYCLE OF 8085:

- The memory read machine cycle is executed by the processor to read a data byte from memory.
- The processor takes 3T states to execute this cycle.
- The instructions which have more than one byte word size will use the machine cycle after the opcode fetch machine cycle.

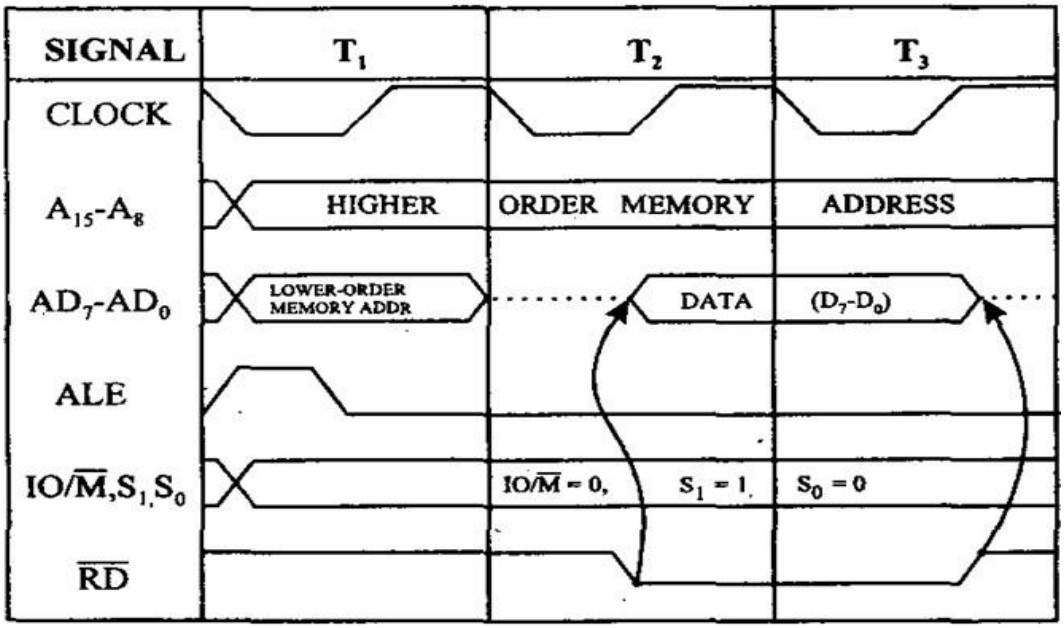


Fig - Timing Diagram for Memory Read Machine Cycle

I/O WRITE CYCLE OF 8085:

- The I/O write machine cycle is executed by the processor to write a data byte in the I/O port or to a peripheral, which is I/O, mapped in the system.
- The processor takes 3T states to execute this machine cycle.

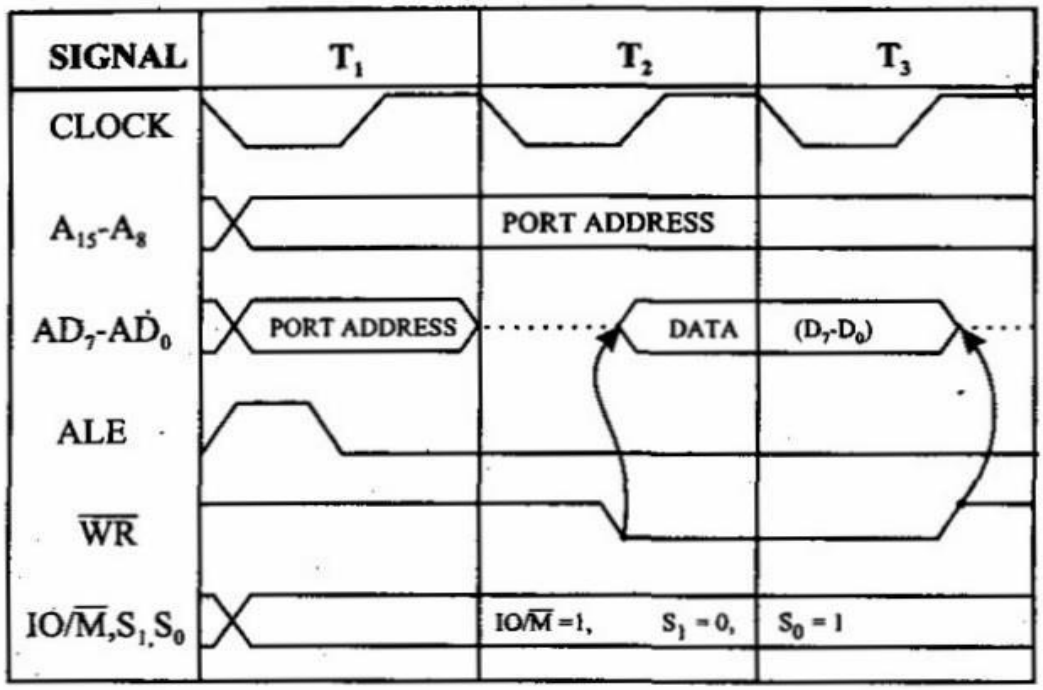


Fig - Timing Diagram for I/O Write Machine Cycle

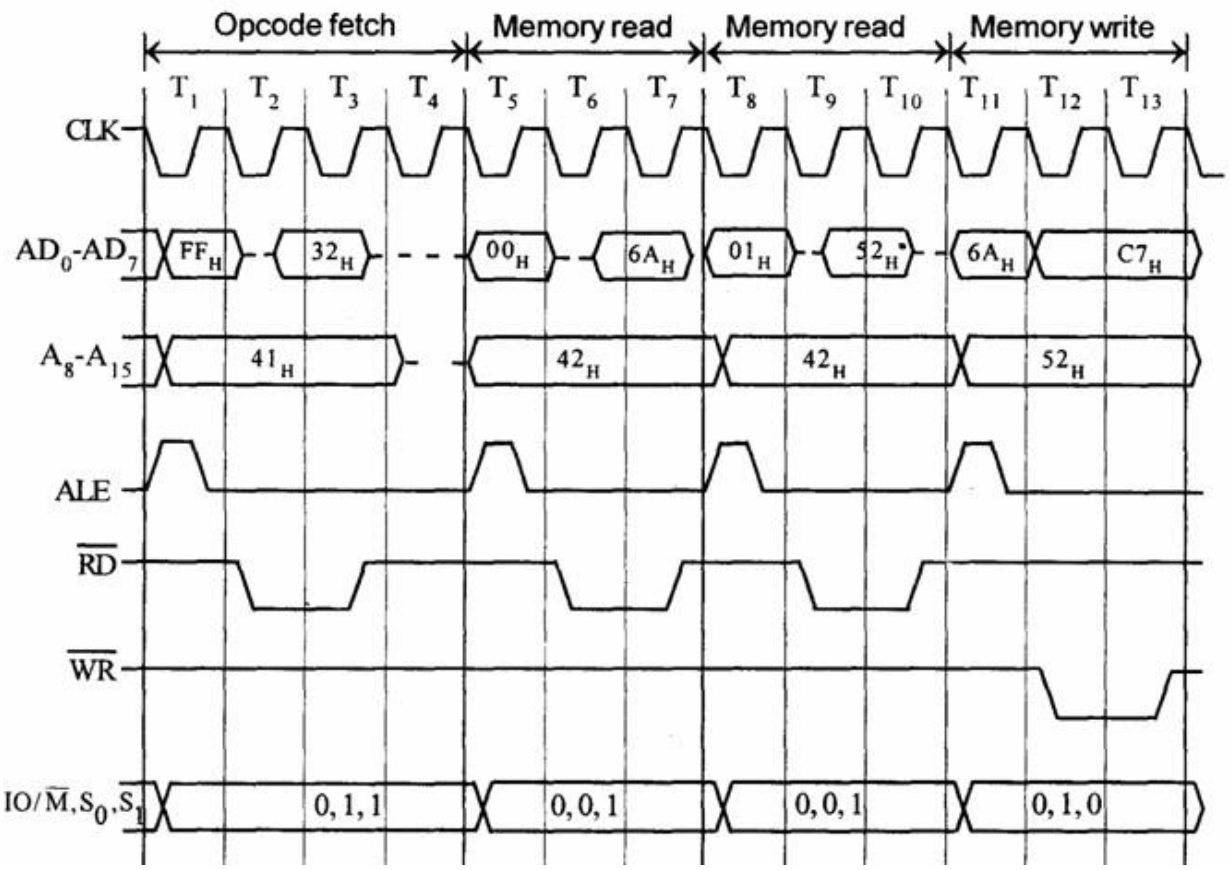
TIMING DIAGRAM OF 8085 INSTRUCTIONS

- The 8085 instructions consist of one to five machine cycles.
- Actually the execution of an instruction is the execution of the machine cycles of that instruction in the predefined order.
- The timing diagram of an instruction ate obtained by drawing the timing diagrams of the machine cycles of that instruction, one by one in the order of execution.

Timing Diagram for STA 526AH

- STA means Store Accumulator -The contents of the accumulator is stored in the specified address (526A).
- The opcode of the STA instruction is said to be 32H. It is fetched from the memory 41FFH (see fig). - OF machine cycle
- Then the lower order memory address is read(6A). - Memory Read Machine Cycle
- Read the higher order memory address (52).- Memory Read Machine Cycle
- The combinations of both the addresses are considered and the content from accumulator is written in 526A. - Memory Write Machine Cycle
- Assume the memory address for the instruction and let the content of accumulator is C7H. So, C7H from accumulator is now stored in 526A.

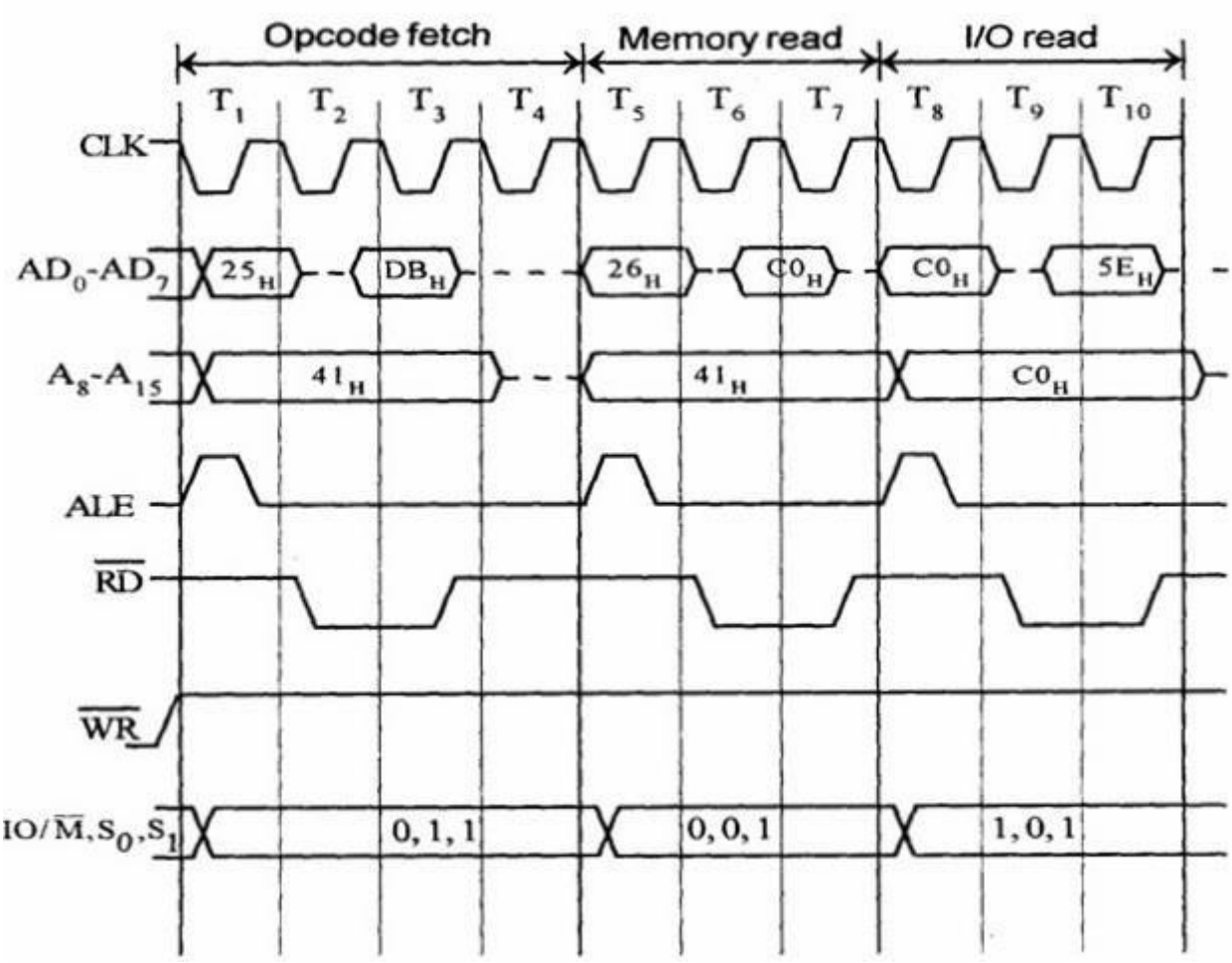
Address	Mnemonics	Opcode
41FF	STA $526A_H$	32_H
4200		$6A_H$
4201		52_H



Timing Diagram for IN C0H

- Fetching the Opcode DBH from the memory 4125H.
- Read the port address C0H from 4126H.
- Read the content of port C0H and send it to the accumulator.
- Let the content of port is 5EH.

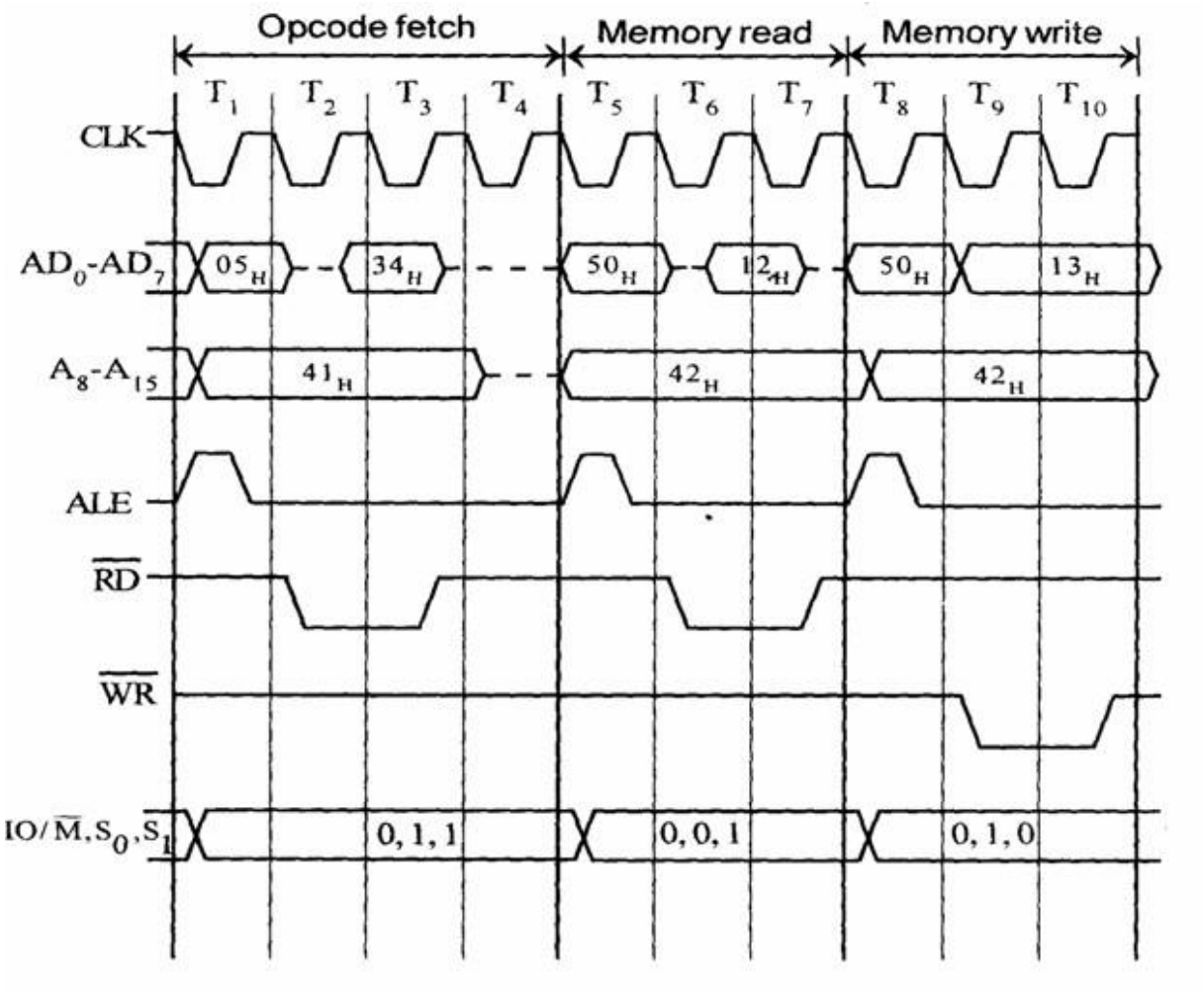
Address	Mnemonics	Opcode
4125	IN $C0_H$	DB_H
4126		$C0_H$



Timing diagram for INR M

- Fetching the Opcode 34H from the memory 4105H. (OF cycle)
- Let the memory address (M) be 4250H. (MR cycle -To read Memory address and data)
- Let the content of that memory is 12H.
- Increment the memory content from 12H to 13H. (MW machine cycle)

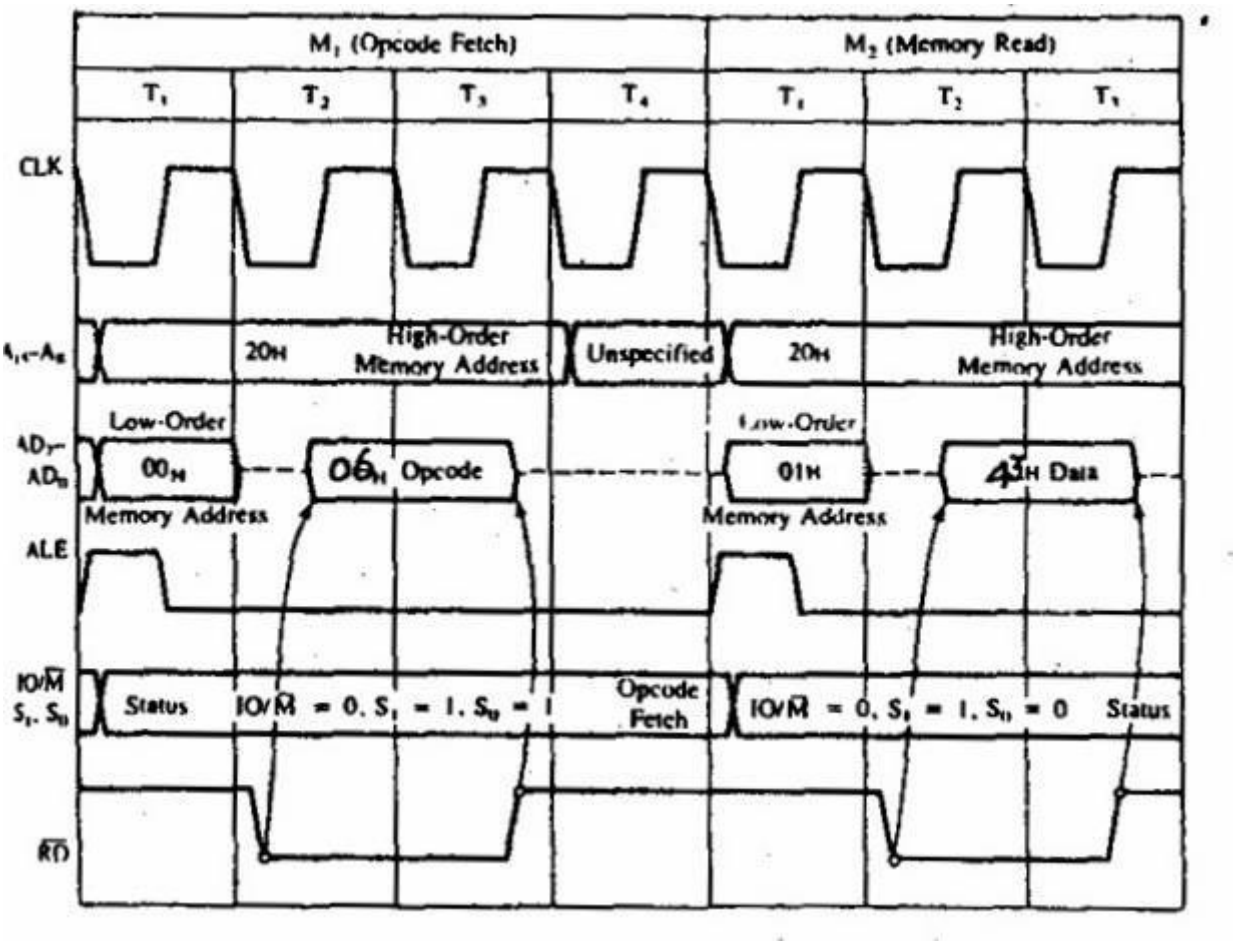
Address	Mnemonics	Opcode
4105	INR M	34_H



Timing diagram for MVI B, 43H

- Fetching the Opcode 06H from the memory 2000H. (OF machine cycle)
- Read (move) the data 43H from memory 2001H. (memory read)

Address	Mnemonics	Opcode
2000	MVI B, 43H	06H
2001		43H



13. EVOLUTION OF 16-BIT & 32-BIT MICROPROCESSOR:

EVOLUTION OF 16-BIT MICROPROCESSORS:

The 16-bit processors: MCS-86 family

1. **8086**

Introduced June 8, 1978

Clock rates:

- 5 MHz with 0.33 MIPS
- 8 MHz with 0.66 MIPS
- 10 MHz with 0.75 MIPS

The memory is divided into odd and even banks; it accesses both banks concurrently to read 16 bits of data in one clock cycle

- Bus width 16 bits data, 20 bits address
- Number of transistors 29,000 at 3 μm

- Addressable memory 1 megabyte
- Up to 10X the performance of 8080

First used in the Compaq Deskpro IBM PC-compatible computers. Later used in portable computing, and in the IBM PS/2 Model 25 and Model 30. Also used in the AT&T PC6300 / Olivetti M24, a popular IBM PC-compatible (predating the IBM PS/2 line).

Used segment registers to access more than 64 KB of data at once, which many programmers complained made their work excessively difficult.

The first x86 CPU.

Later renamed the iAPX 86[4]

2. 8088

Introduced June 1, 1979

Clock rates:

- 4.77 MHz with 0.33 MIPS
- 8 MHz with 0.66 MIPS[3]

Internal architecture 16 bits

External bus Width 8 bits data, 20 bits address

Number of transistors 29,000 at 3 μm

Addressable memory 1 megabyte

Later renamed the iAPX 88

3. 80186

Introduced 1982

Clock rates

- 6 MHz with > 1 MIPS

Included two timers, a DMA controller, and an interrupt controller on the chip in addition to the processor (these were at fixed addresses which differed from the IBM PC, although it was used by several PC compatible vendors such as Australian company Cleveland).

Added a few opcodes and exceptions to the 8086 design; otherwise identical instruction set to 8086 and 8088

BOUND, ENTER, LEAVE

INS, OUTS

IMUL imm, PUSH imm, PUSHA, POPA

RCL/RCR/ROL/ROR/SHL/SHR/SAL/SAR reg,imm

Address calculation and shift operations are faster than 8086

Later renamed the iAPX 186

4. 80188

A version of the 80186 with an 8-bit external data bus

Later renamed the iAPX 188

5. 80286

Introduced February 2, 1982

Clock rates:

- 6 MHz with 0.9 MIPS
- 8 MHz, 10 MHz with 1.5 MIPS
- 12.5 MHz with 2.66 MIPS
- 16 MHz, 20 MHz and 25 MHz available.

Bus width: 16 bits data, 24 bits address.

EVOLUTION OF 32-BIT MICROPROCESSORS:

32-bit processors: the non-x86 microprocessors

- iAPX 432
- i960 aka 80960
- i860 aka 80860
- XScale

32-bit processors: the 80386 range

- 80386DX
- 80386SX
- 80376
- 80386SL
- 80386EX

32-bit processors: the 80486 range

- 80486DX
- 80486SX
- 80486DX2
- 80486SL
- 80486DX4

32-bit processors: P5 microarchitecture

- Original Pentium
- Pentium with MMX Technology

32-bit processors: P6/Pentium M microarchitecture

- Pentium Pro
- Pentium II
- Celeron (Pentium II-based)
- Pentium III
- Pentium II and III Xeon
- Celeron (Pentium III Coppermine-based)
- Celeron (Pentium III Tualatin-based)
- Pentium M
- Celeron M
- Intel Core
- Dual-Core Xeon LV

32-bit processors: NetBurst microarchitecture

- Pentium 4
- Xeon
- Mobile Pentium 4-M
- Pentium 4 EE
- Pentium 4E
- Pentium 4F